MATH 112 Calculus 2
Project Module
Spring 2025

April 1, 2025

Preface

Welcome to the Calculus 2 Project module, a companion resource designed for the MATH 112 Calculus II course at Khalifa University for the Spring Term 2025. This module aims to provide students with practical tools and examples to enhance their understanding of calculus concepts using the SymPy library in Python.

SymPy is a powerful Python library for symbolic mathematics, enabling users to perform algebraic manipulations, calculus operations, and more. Throughout this module, we will explore various topics such as symbolic computation, calculus, algebra, discrete mathematics, and geometry, all within the context of the Calculus II curriculum.

This module is intended to be used alongside the main reference textbook for the course:

Calculus: Early Transcendental Functions by Robert T. Smith, Roland B. Minton and Ziad A. T. Rafhi, McGraw-Hill, 5th Edition, 2018, ISBN: 9781526869968

All mathematical notations and conventions used here are consistent with those in the main reference.

We hope this module serves as a valuable resource in your journey through Calculus II, providing you with the computational tools needed to tackle complex mathematical problems.

Calculus 2 Project Teaching Assistants:
Abrari Noor, Ade Bayu, Defri Ahmad, Ismaila, Muhammad Ridwan

Contents

Chapter 1

Introduction to Sympy Library

SymPy is a Python library for symbolic mathematics. It provides tools for:

```
- Symbolic computation: Manipulating mathematical expressions symbolically.
- Calculus: Differentiation, integration, limits, series expansion.
- Algebra: Solving equations, simplifying expressions, matrix operations.
- Discrete mathematics: Combinatorics, number theory.
- Geometry: Geometric calculations, plotting.
```

Here we use Python to perform operations common in calculus by utilizing the SymPy library. In depth explanation can be found in its official documentation.

1.1 Importing SymPy library

The syntax for importing a library in Python is as follows.

[4]:
```
import sympy as smp
```

In the syntax above, the library name is sympy and the alias we create is smp. An alias is an easy-to-remember abbreviation for calling a library.

1.2 Declaring Variables and Functions

In the integral operation, we perform the integration on the integrand function with respect to certain variables. Declaring variables and functions at the beginning in Python code will be beneficial in creating integration code in the next step.

1.2.1 Declaring Variables

We utilize the `symbols` from `SymPy` library to declare variables.
Syntax: variable_name = `smp.symbols`("variable_name", options).

Example 1.1. Declaring single variable x as real number.

[5]:
```
x = smp.symbols("x", real=True)
```

Example 1.2. Declaring multiple variables x, y, and z as real number.

[6]:
```
x, y, z = smp.symbols("x, y, z", real=True)
```

Example 1.3. Declaring single variable n as positive integer.

[7]:
```
n = smp.Symbol("n", integer = True, positive = True) # defines 'n' as a␣
 ↪positive integer
```

Example 1.4. Declaring single variable m as with value $\sqrt{5}$.

[8]:
```
m = smp.sqrt(5)
display(m)
```

$\sqrt{5}$

Use `.evalf()` after variable name to get decimal value of the variable.

[9]:
```
print(f"m = {m} = {m.evalf()}")
```

```
m = sqrt(5) = 2.23606797749979
```

1.2.2 Declaring Functions

We utilize the `Function` from `SymPy` library to declare functions.
Syntax: function_name = library_alias.Function("Function_name")(variable_name).

Example 1.5. Defining an unknown function, the name of function is f and the variable is x.

[10]:
```
f = smp.Function("f")(x)

display(f)
print(f)
```

$f(x)$

```
f(x)
```

Example 1.6. Declaring explicit function $g(x) = x^2 + \sin x$. In this function, the name of function is g and the variable is x.

[11]:
```
g = x**2 + smp.sin(x)

display(g)
```

$x^2 + \sin(x)$

The table below gives the syntax for the most used mathematical functions in `sympy`.

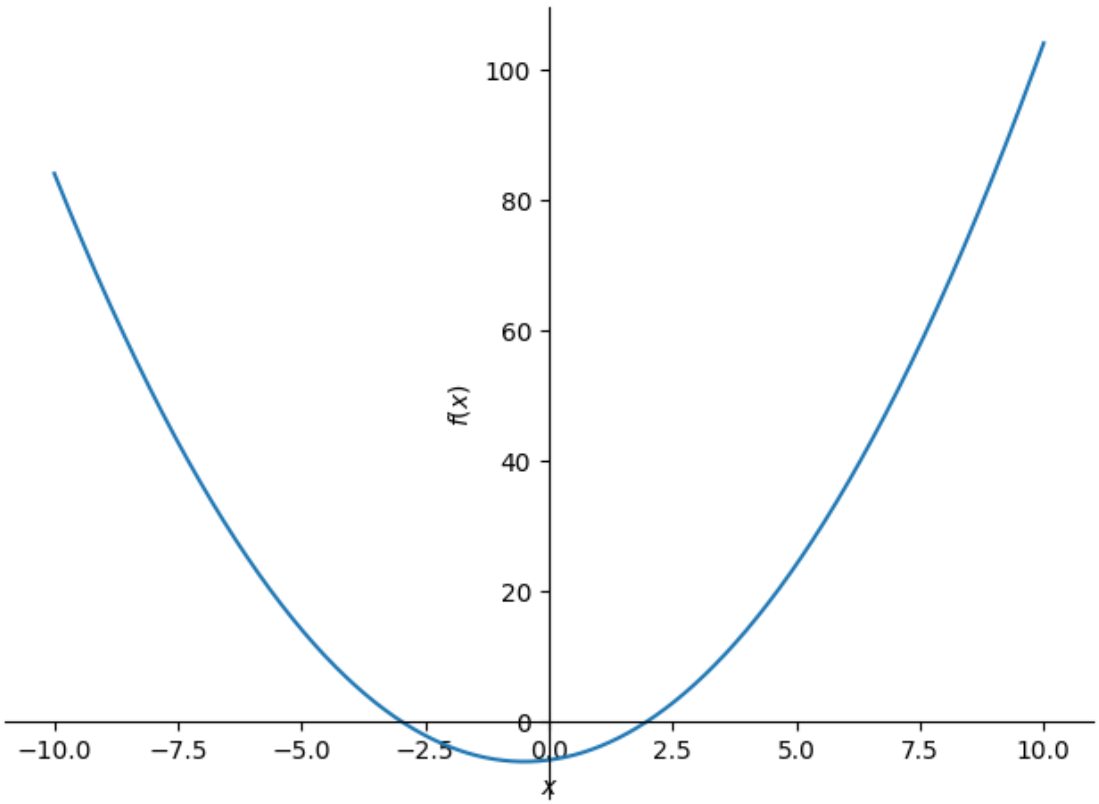
Mathematical Expression	Sympy	Mathematical Expression	Sympy
$\sin(x)$	`sin(x)`	$\cot^{-1}(x)$	`acot(x)`
$\cos(x)$	`cos(x)`	$\sec^{-1}(x)$	`asec(x)`
$\tan(x)$	`tan(x)`	$\csc^{-1}(x)$	`acsc(x)`
$\cot(x)$	`cot(x)`	e^x	`exp(x)`
$\sec(x)$	`sec(x)`	$\ln(x)$	`ln(x)`
$\csc(x)$	`csc(x)`	$\log_y(x)$	`log(x,y)`
$\sin^{-1}(x)$	`asin(x)`	x^n	`x**n`
$\cos^{-1}(x)$	`acos(x)`	$\sqrt{(x)}$	`sqrt(x)`
$\tan^{-1}(x)$	`atan(x)`	$\sqrt[n]{(x)}$	`root(x,n)`

1.3 Plotting Functions

There are many libraries that can be used for plotting for example, `sympy` and `matplotlib`. In `sympy` we use the command `plot` for plotting functions of a single variable.
Syntax: `smp.plot`(function)

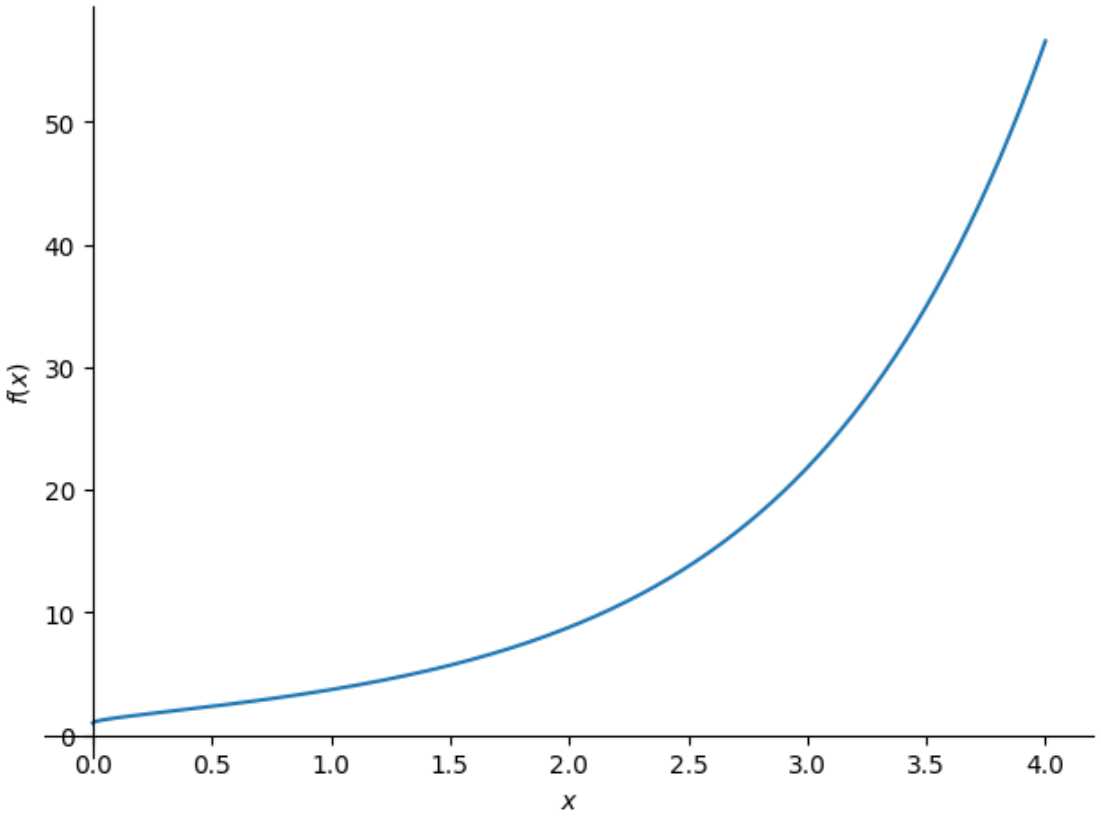
Example 1.7. Plot function $f(x) = x^2 + x - 6$

[12]:
```
smp.plot(x**2+x-6);
```

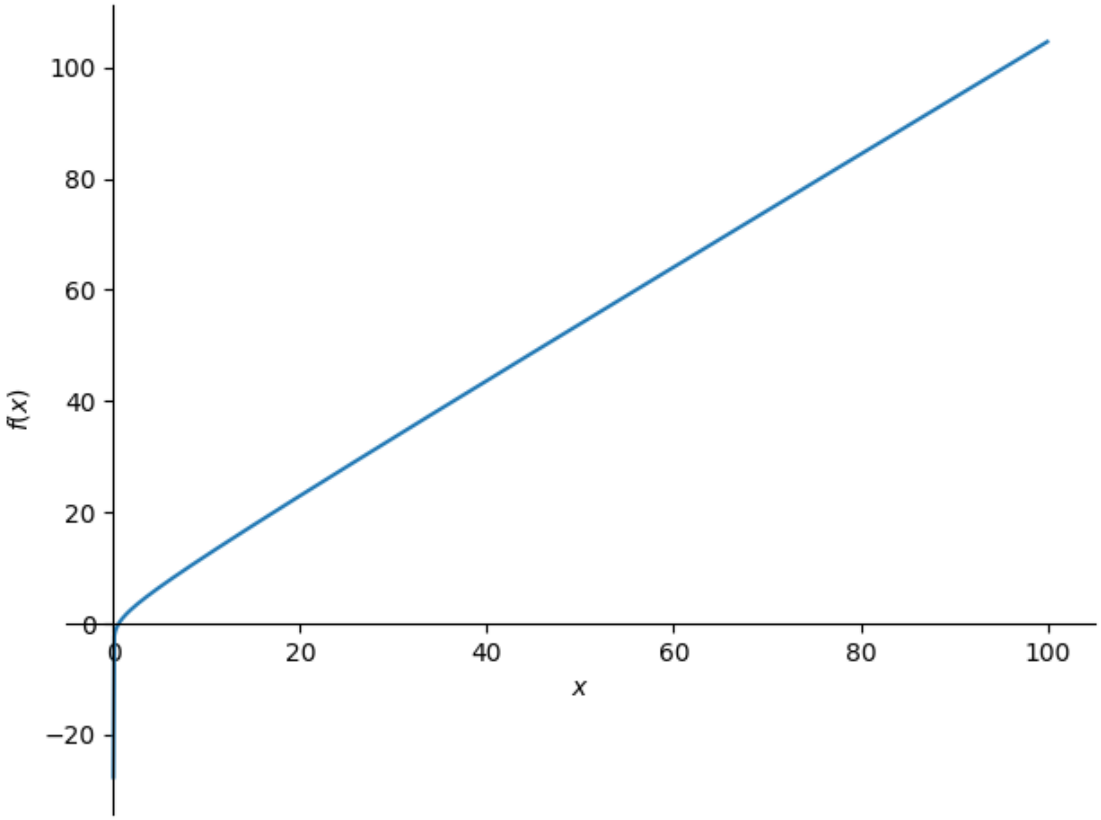


Example 1.8. Plot function $g(x) = x^3 - 3x^2 + 3x - 1$ on domain $0 \leq x \leq 4$.

[13]:
```
smp.plot(smp.exp(x)+smp.sqrt(x), (x,0,4));
```

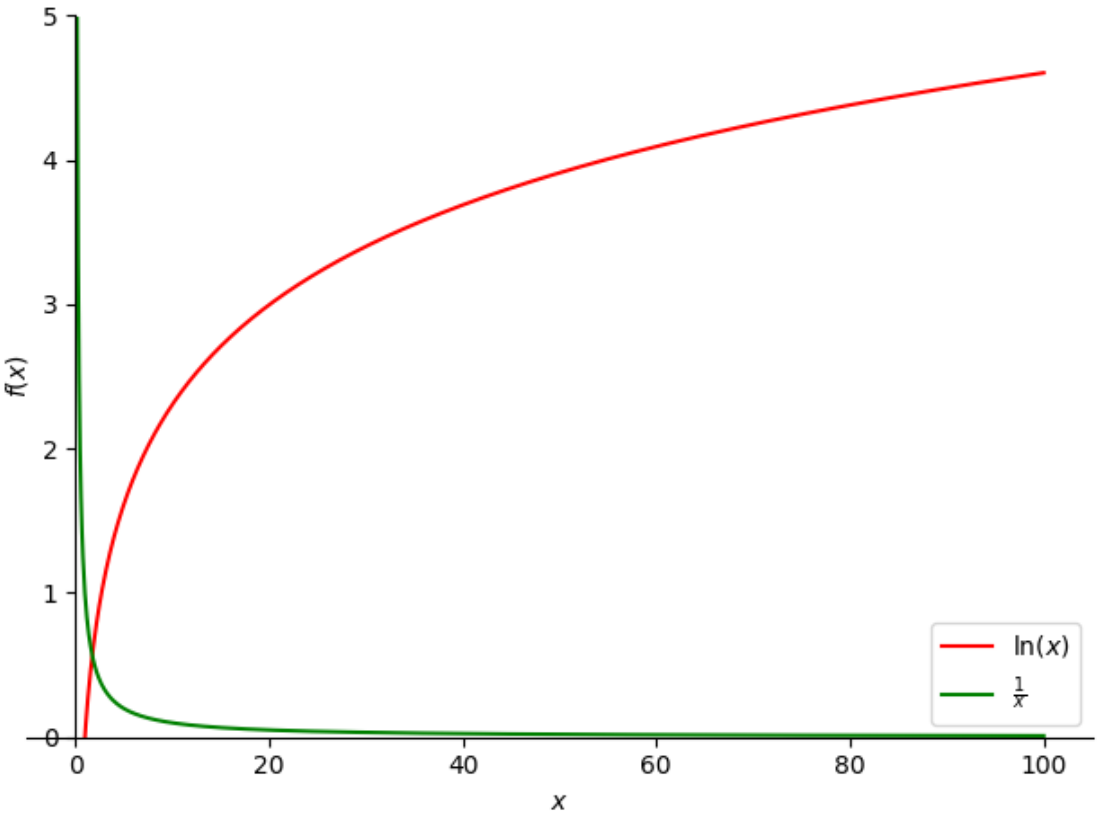


[14]:
```
smp.plot(x+smp.ln(x), (x,1e-12,100));
```



You can further customize the details of the figure. Below is an example of combining two plots into one figure and modifying the default plot settings. For detail, please consult the documentation .

[15]:
```
# Plot the natural logarithm function ln(x) from x=1e-5 to x=100
# Set the label, line color, line width, legend, and y-axis limits
fig1 = smp.plot(smp.ln(x), (x, 1e-5, 100), label=r"$\ln(x)$", line_color="red",␣
 ↪linewidth=4, legend=True, ylim=(0, 5), show=False)

# Plot the function 1/x from x=1e-5 to x=100
# Set the label, line color, line width, and y-axis limits
fig2 = smp.plot(1/x, (x, 1e-5, 100), label=r"$\frac{1}{x}$",␣
 ↪line_color="green", linewidth=4, ylim=(0, 5), show=False)

# Append the second plot (fig2) to the first plot (fig1)
fig1.append(fig2[0])

# Display the combined plot
fig1.show()
```


5
4
3
2
1
0
f(x)
0
20
40
60
80
100
x
ln(x)
1/x

Chapter 2

Integrals

To compute an integral, use the `integrate` from `SymPy` library. There are two kinds of integrals, definite and indefinite. In this section, we will explore various examples that demonstrate how to compute both definite and indefinite integrals using the `SymPy` library. These examples will cover some of the integral techniques discussed in Section 6, including integration by parts, trigonometric integrals, and the use of partial fractions.

2.1 Indefinite Integral

To compute an indefinite integral, that is, an antiderivative, or primitive, just pass the variable after the function expression or formula. Syntax: `smp.integrate`(function formula, variable)

Example 2.1. Evaluate $\int (x^2 + 1)\, dx$

[16]:
```
smp.integrate(x**2+1, x)
```

[16]:
$$\frac{x^3}{3} + x$$

Example 2.2. Evaluate $\int x \ln x \; dx$.

This integral is usually solved by partial integration.

[17]:
```
smp.integrate(x*smp.ln(x), x)
```

[17]:
$$\frac{x^2 \log(x)}{2} - \frac{x^2}{4}$$

In this result, the $\log x = \ln x$

Example 2.3. Evaluate $\int \frac{x^2}{\sqrt{16 - x^2}}\, dx$.

This integral involves trigonometric techniques of integration.

[18]:
```
f = x**2/smp.sqrt(16-x**2)  # Declaring function
display(f)   # Check whether the displayed function is correct or still wrong.
```

```
smp.integrate(f, x)
```

[18]:
$$\frac{x^2}{\sqrt{16-x^2}}$$
$$-\frac{x\sqrt{16-x^2}}{2}+8\,\mathrm{asin}\left(\frac{x}{4}\right)$$

In this result, $\mathrm{asin}(x/4)=\sin^{-1}(x/4)$

Example 2.4. Evaluate $\displaystyle\int \frac{x^3+x+2}{x^2+2x-8}\,dx.$

This is an example of integration of rational functions using partial fractions.

[19]:
```
g = (x**3+x+2)/(x**2+2*x-8)
display(g)
smp.integrate(g, x)
```

[19]:
$$\frac{x^3+x+2}{x^2+2x-8}$$
$$\frac{x^2}{2}-2x+2\log(x-2)+11\log(x+4)$$

Example 2.5. Evaluate $\displaystyle\int x^5e^x\sin x\;dx.$

This is a very hard integral. Don't try this by hand unless you have plenty of time and patience. However, we can solve this integral easily by a computer code.

[20]:
```
h = x**5*smp.exp(x)*smp.sin(x)
display(h)
smp.integrate(h, x)
```

[20]:
$$x^5e^x\sin(x)$$
$$\frac{x^5e^x\sin(x)}{2}-\frac{x^5e^x\cos(x)}{2}+\frac{5x^4e^x\cos(x)}{2}-5x^3e^x\sin(x)-5x^3e^x\cos(x)+15x^2e^x\sin(x)-15xe^x\sin(x)+15xe^x\cos(x)-15e^x\cos(x)$$

[21]:
```
# Define the function g
g = smp.sqrt(x**2 - 16) / x
display(g)
# Integrate the function g with respect to x
smp.integrate(g, x)
```

[21]:
$$\frac{\sqrt{x^2-16}}{x}$$
$$\begin{cases} -\frac{ix}{\sqrt{-1+\frac{16}{x^2}}}-\frac{4i|x|\,\mathrm{acosh}\left(\frac{4}{|x|}\right)}{x}+\frac{2\pi|x|}{x}+\frac{16i}{x\sqrt{-1+\frac{16}{x^2}}} & \text{for } \frac{1}{x^2}>\frac{1}{16} \\ \frac{x}{\sqrt{1-\frac{16}{x^2}}}+\frac{4|x|\,\mathrm{asin}\left(\frac{4}{|x|}\right)}{x}-\frac{16}{x\sqrt{1-\frac{16}{x^2}}} & \text{otherwise} \end{cases}$$

2.2 Definite Integral

Syntax: `smp.integrate`(function formula, (variable, lower limit, upper limit))

Example 2.6. Evaluate $\int_0^3 (x^2+1)\ dx$

[22]:
```
smp.integrate(x**2+1, (x, 0, 3))
```

[22]:
$$12$$

Example 2.7. Evaluate $\int_0^1 x^2 e^{3x}\ dx.$

This integral is an example of a definite integral using integration by parts method.

[23]:
```
f = x**2*smp.exp(3*x)  # Declaring function
display(f)  # Check whether the displayed function is correct or still wrong.
smp.integrate(f, (x,0,1))
```

$$x^2 e^{3x}$$

[23]:
$$-\frac{2}{27}+\frac{5e^3}{27}$$

Example 2.8. Evaluate $\int_0^{\pi/4} \tan^4 x \sec^4 x\ dx.$

This integral is an example of a definite integral using trigonometric techniques of integration.

[24]:
```
g = smp.tan(x)**4*smp.sec(x)**4  # Declaring function
display(g)  # Check whether the displayed function is correct or still wrong.
smp.integrate(g, (x,0,smp.pi/4)) # We use smp.pi to express pi
```

$$\tan^4(x)\sec^4(x)$$

[24]:
$$\frac{12}{35}$$

Example 2.9. Evaluate $\int_1^\infty \frac{3}{x^2}\ dx.$

Note that this integral is an improper integral because the upper limit goes to infinity.

[25]:
```
h = 3/x**2
display(h)
smp.integrate(h, (x,1,smp.oo))  # We use smp.oo (double small "oh") to express␣
↪infinity.
```

$$\frac{3}{x^2}$$

[25]:
$$3$$

Since we obtain a real and finite value, then the integral is converges with $\int_1^\infty \frac{3}{x^2}\ dx = 3.$

Example 2.10. Evaluate $\int_{-1}^2 \frac{3}{x^2}\ dx.$

[26]:
```
h = 3/x**2
display(h)
smp.integrate(h, (x,-1,2))  # Integral of h from -1 to 2
```

$$\frac{3}{x^2}$$

[26]:
$$\infty$$

Can you explain why the integral is diverge ?

Chapter 3

Parametric Equations and Polar Coordinates

3.1 Plane Curves and Parametric Equations

3.1.1 Sketching Plane Curve

General Procedure;

- Declare x and y as symbols. This requires the symbols or Symbol functions of sympy package
- Plot the implicit curve given in the question, to verify the symmetry. To do this symbolically, we need to import the `plot_implicit` function of the `sympy.plotting` package from the `sympy` library. For the implicit plot, the equation of the curve is written in the form of $f(x, y) = 0$, that is the right-hand side equals zero.

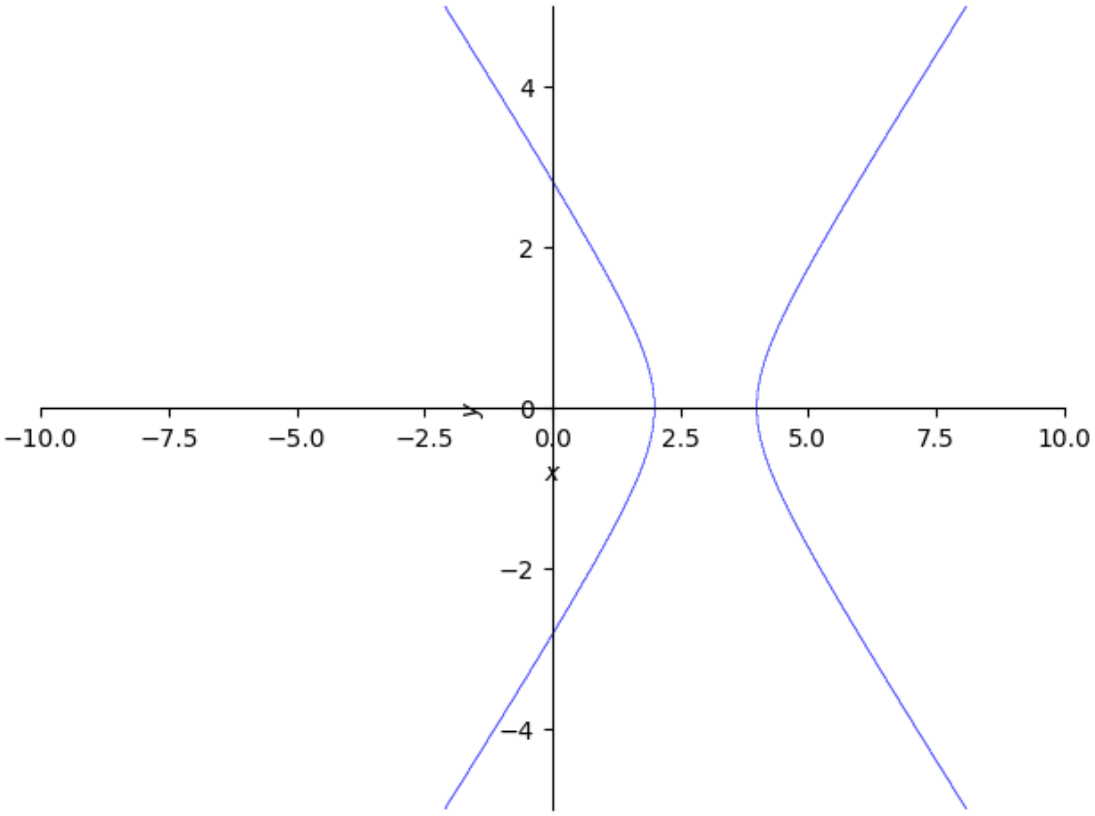
Example 3.1. Sketch the curve of the following equation

1. $(x-3)^2 - (y+2)^2 = 1$
2. $x^{5/2} + y^{5/2} - 4 = 0$

[27]:
```
import sympy as smp
```

[28]:
```
x,y=smp.symbols("x y", real=True) #defines x and y as symbolic variables
```

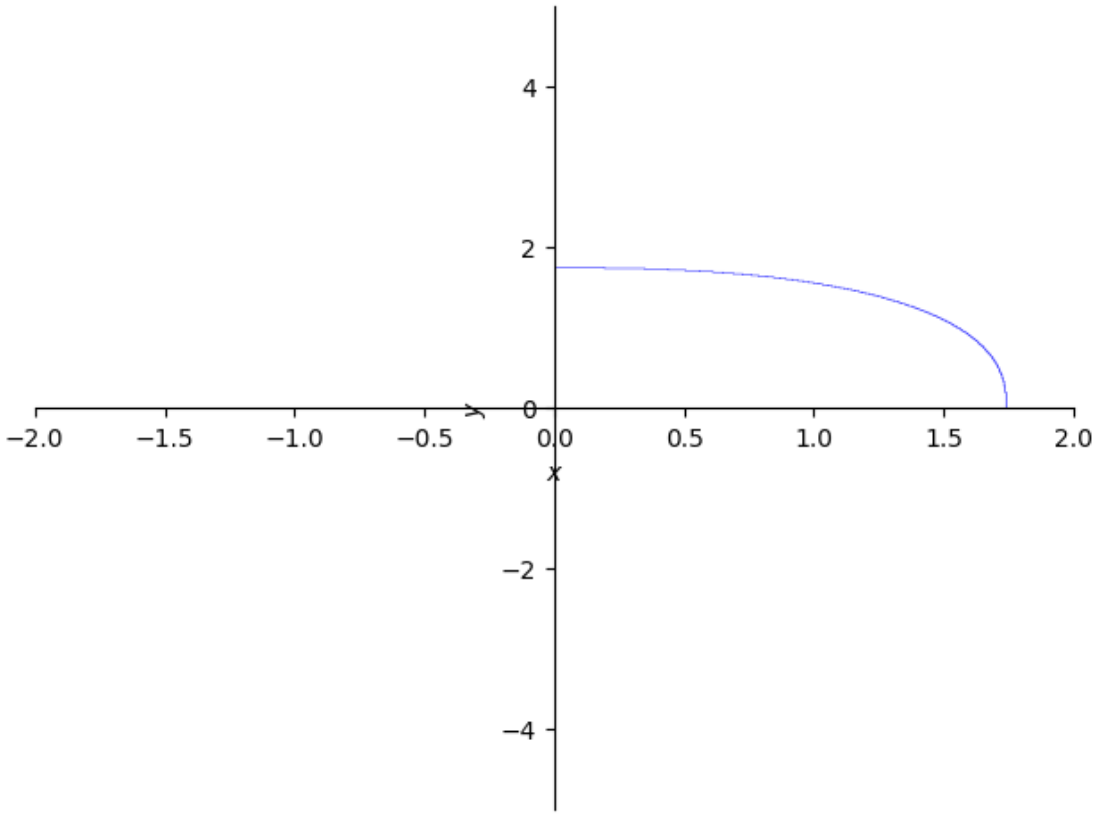
[29]:
```
smp.plot_implicit((x-3)**(2)-(y)**(2)-1, (x,-10,10))
# The first argument, (x-3)**(2)-(y)**(2)-1, is the equation to be plotted,
# The second argument, (x,-10,10), specifies the range of x-values for the␣
 ↪plot, from -10 to 10.
```



[29]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b9a50d880>
```

Can you find something strange between the code and the result of the following code? Can you explain why?

[30]:
```
smp.plot_implicit(x**(5/2)+y**(5/2)-4, (x,-2,2))
```



[30]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b99d4fc40>
```

3.1.2 Sketching Parametric Equation

To plot a parametric equation sometimes we need to import package from `sympy.plotting` and choose kind of plot that you want. For instance using `plot_parametric` .

Can you differentiate each kind of plotting method in sympy.plotting?

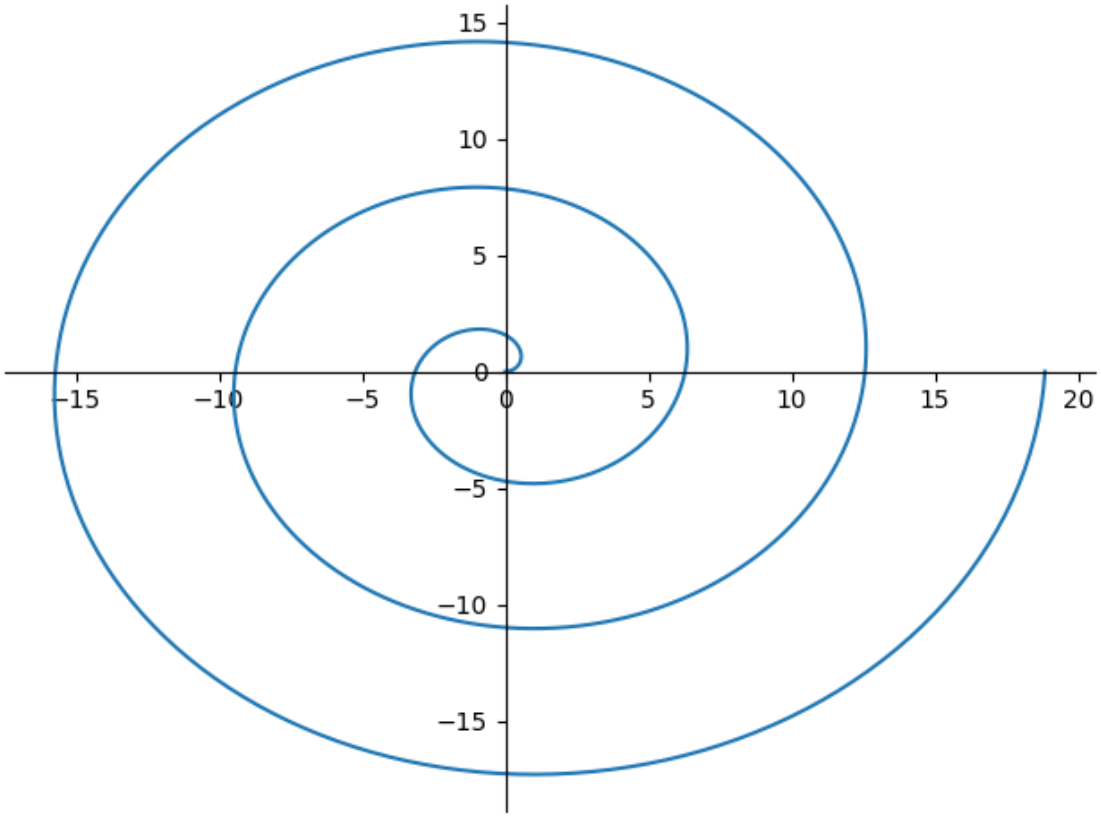
Example 3.2. Plot the plane curve defined by the following parametric equations

$$x = t\cos(t)$$
$$y = t\sin(t)$$

for $t = 0$ to 6π

[31]:
```
t=smp.symbols('t') #defines 't' as a symbolic variable, representing the␣
↪parameter for the parametric equations.

x=t*smp.cos(t) # equation for x in terms of t.
y=t*smp.sin(t) # equation for y in terms of t
```

```
smp.plot_parametric(x,y,(t,0,6*smp.pi)) # x and y are the parametric equations,␣
↪(t, 0, 6*pi) specifies that the parameter t ranges from 0 to 6π.
```



[31]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x18b9a06aee0>

3.2 Calculus and Parametric Equations

3.2.1 Derivative of Parametric Equation

In this section, we will explore how to compute the derivatives of parametric equations using the `smp.diff` function from the SymPy library. The `smp.diff` function allows us to differentiate expressions with respect to a given variable. We will demonstrate this process with an example.

Example 3.3. Find dx/dt, dy/dt, and dy/dx of the curves defines with the following parameteric equations:

$$x = \sin(t), \qquad y = t^2$$

[32]:
```
t=smp.symbols('t') # This line defines t as a symbolic variable, representing␣
↪the parameter.
x=smp.sin(t)  # define the parametric equations for x
y=t**2 # define the parametric equations for y

dxdt = smp.diff(x,t) # Calculates the derivative of x with respect to t
print("Derivative of x with respect to t is",dxdt)
display(dxdt)

dydt = smp.diff(y,t)  # Calculates the derivative of y with respect to t
print("Derivative of y with respect to t is",dydt)
display(dydt)

dydx = dydt/dxdt # Calculates the derivative of y with respect to x by dividing␣
↪dy/dt by dx/dt
print("Derivative of y with respect to x (still in t) is",dydx)
display(dydx)
```

```
Derivative of x with respect to t is cos(t)
```

$\cos(t)$

```
Derivative of y with respect to t is 2*t
```

$2t$

```
Derivative of y with respect to x (still in t) is 2*t/cos(t)
```

$$\frac{2t}{\cos(t)}$$

3.2.2 Plot Tangent Line/ Slope

Procedure;

1. Define the parametric equations: Express the curve as x = f(t) and y = g(t).
2. Find the point of tangency: Determine the values of x and y at the given parameter value t.
3. Calculate derivatives using `diff`: Compute the derivatives dx/dt and dy/dt.
4. Calculate dy/dx: Divide dy/dt by dx/dt to find the slope of the tangent line.
5. Find the slope at the given point: Substitute the parameter value t into dy/dx to get the slope m.
6. Find the equation of the tangent line: Use the point-slope form $y - y_0 = m(x - x_0)$, where (x_0, y_0) is the point of tangency.
7. Plot the curve and tangent line: Create a plot that displays both the parametric curve and the tangent line.

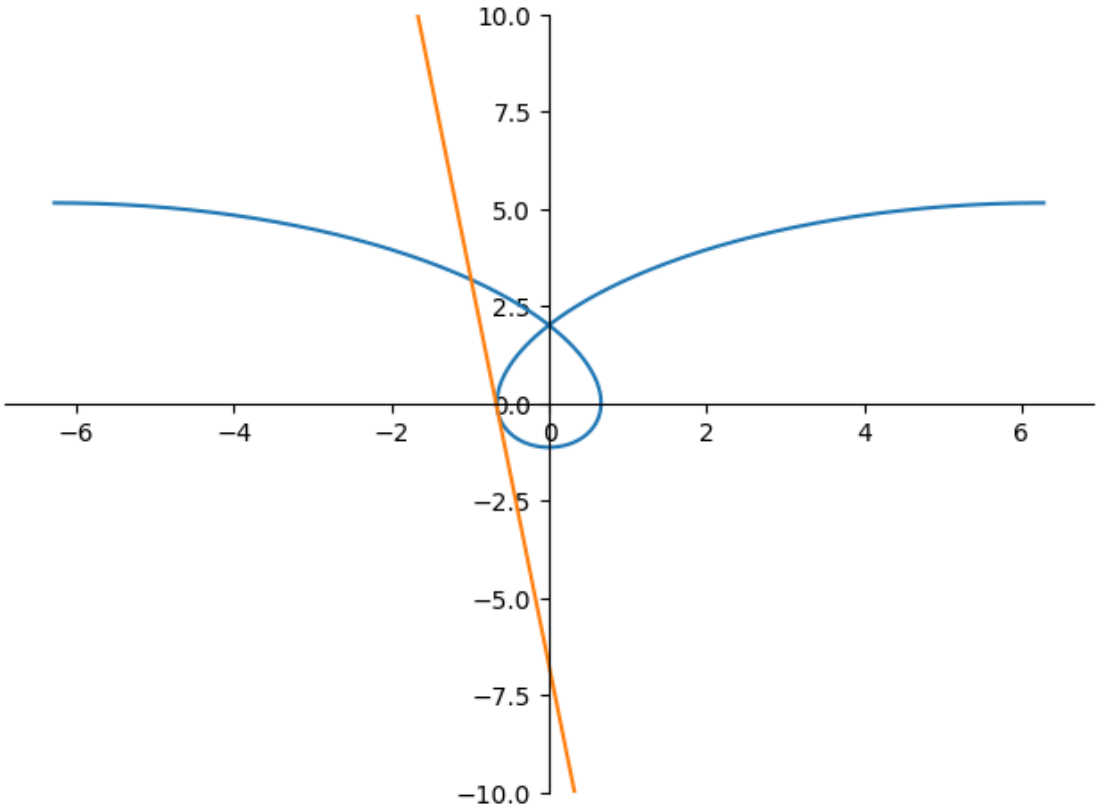
Example 3.4. Plot the curve and the slope of the curve below

$$x = 2t - \pi\sin(t) \quad y = 2 - \pi\cos(t)$$

at $t = \pi/4$

[33]:
```
t =smp.symbols('t',real=True)

# Define the parametric equations for x and y
x=2*t-smp.pi*smp.sin(t)
y=2-smp.pi*smp.cos(t)

# Calculate the values of x and y at t = pi/2
x_0 = x.subs(t,smp.pi/4)
y_0 = y.subs(t,smp.pi/4)

# Calculate the derivatives of x and y with respect to t
dxdt = smp.diff(x,t)
dydt = smp.diff(y,t)


dydx = dydt/dxdt # Calculates the derivative of y with respect to x

m = dydx.subs(t,smp.pi/4); #Calculates the slope of the tangent line at x_0,y_0

curve = smp.plot_parametric(x,y,(t,-smp.pi,smp.pi), show=False, ylim =␣
↪(-10,10)) # Creates a parametric plot of the curve but doesn't show it yet

a = smp.minimum(x, t, smp.Interval(-smp.pi,smp.pi))
b = smp.maximum(x, t, smp.Interval(-smp.pi,smp.pi))

xm=smp.symbols('x', real=True) #Defines symbolic variables for the tangent line␣
↪equation
TangenLine = m*(xm-x_0)+y_0 # Defines the equation of the tangent line using␣
↪the point-slope form
print("Equation of slope:",TangenLine.evalf())

TgnLine = smp.plot(TangenLine,(xm,a,b), show=False) #Creates a plot of the␣
↪tangent line but doesn't show it yet

curve.extend(TgnLine) # Adds the tangent line plot to the curve plo
curve.show() # Displays the combined plot of the curve and the tangent line
```

```
Equation of slope: -10.0317319891192*x - 6.74853915649745
```

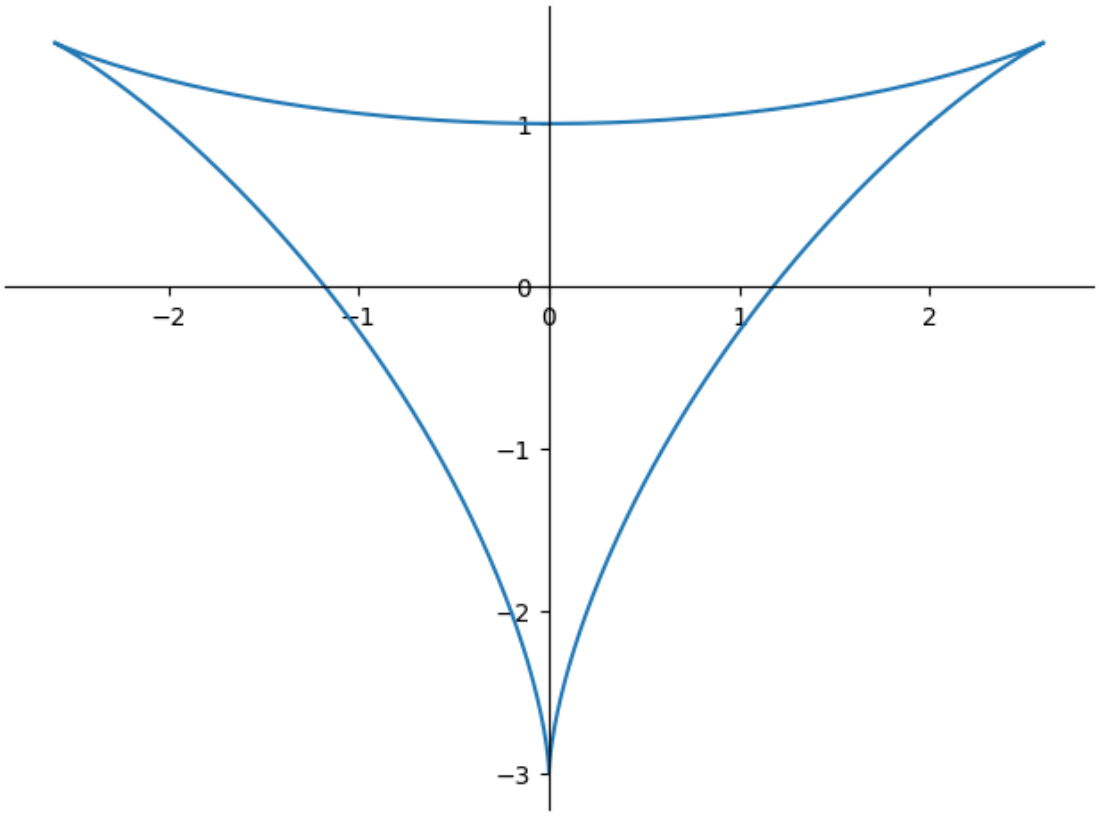


Example 3.5. Find the area enclosed by the curve

$$x = 2\cos(t) + \sin(2t), \qquad y = 2\sin(t) + \cos(2t)$$

We plot the parametric equation:

[34]:
```
t = smp.symbols('t', real=True) # Defines t as a symbolic variable representing␣
↪the parameter

#Define the parametric equations for the curve
x=2*smp.cos(t)+smp.sin(2*t)
y=2*smp.sin(t)+smp.cos(2*t)

smp.plot_parametric(x,y,(t,0,2*smp.pi)) #Plots the parametric curve to␣
↪visualize its shape.
```



[34]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b9a09e130>
```

Then calculate the integral of

$$A = \int_{t_0}^{t_1} x(t)y'(t)dt = -\int_{t_0}^{t_1} y(t)x'(t)dt$$

[35]:
```
dxdt = smp.diff(x,t) #Calculates the derivative of x with respect to t.
Area = smp.integrate(-y*dxdt,(t,0,2*smp.pi)) # Calculates the area enclosed by␣
 ↪the curve

print("The area under the curve C and x axis is")
print(Area.evalf())
```

```
The area under the curve C and x axis is
6.28318530717959
```

3.3 Polar Coordinates

Polar Plot

sympy does not support the polar plot, but you can still use sympy package by a little trick i.e. regard them as parametric plot and defining $x = r \cdot \cos(\theta)$ and $y = r \cdot \sin(\theta)$.

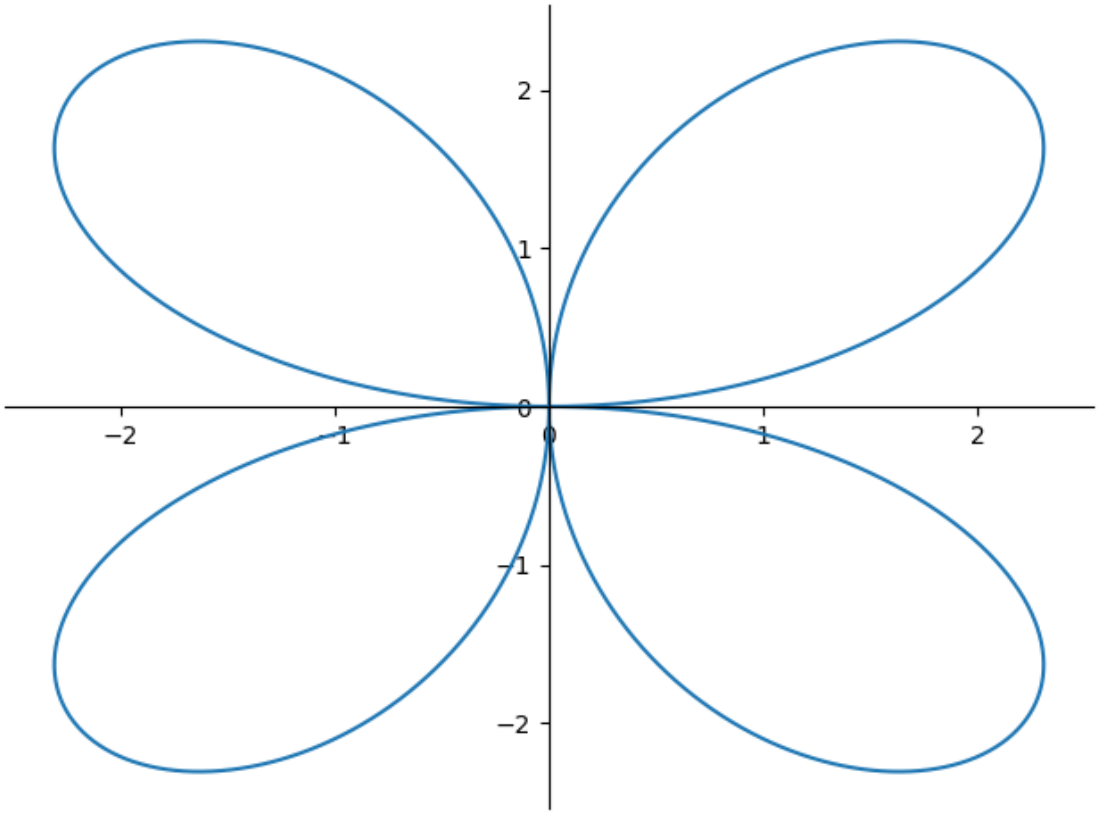
let see the following example

Example 3.6. Plot the following polar equation

$$r = 3\sin(2\theta)$$

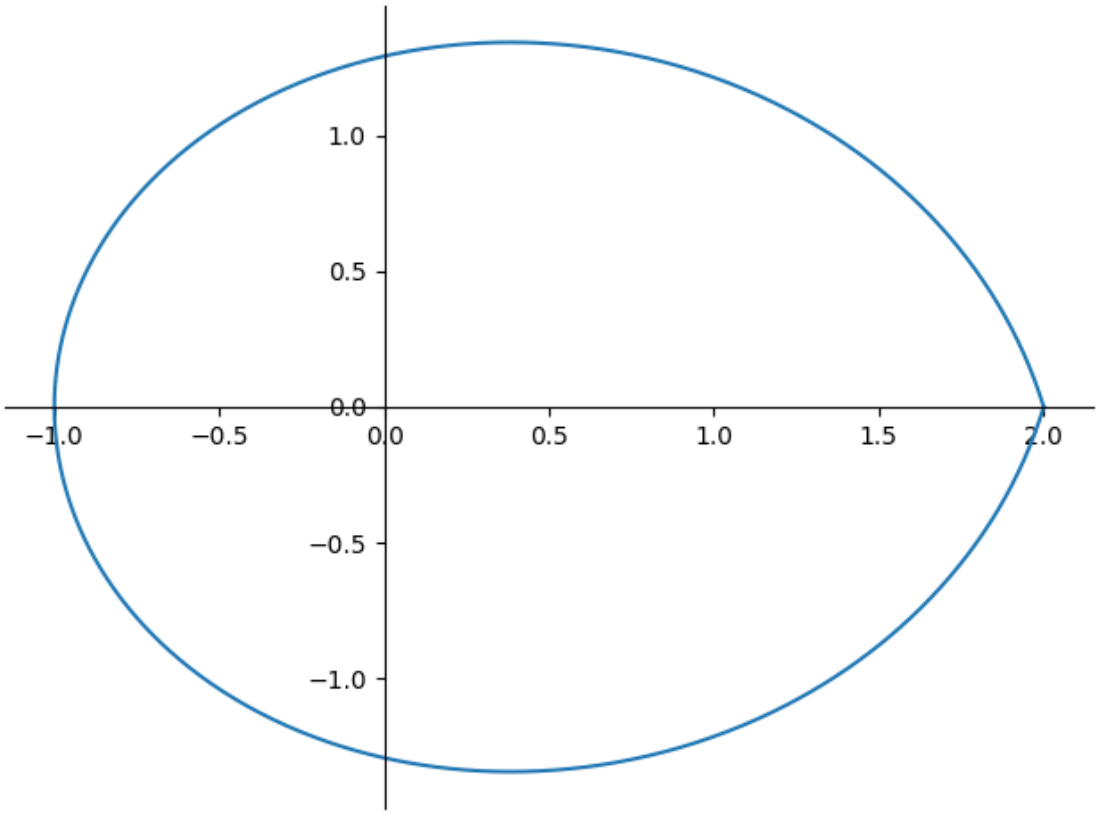
for $\theta = 0..2\pi$

[36]:
```
t = smp.symbols('t', real=True)
r=3*smp.sin(2*t) # the polar equation, where r is the radius and t is the angle.
smp.plot_parametric(r*smp.cos(t),r*smp.sin(t),(t,0,2*smp.pi)) #plot using␣
 ↪parameteric equations
```



[36]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b99eb1a00>
```

[37]:
```
t = smp.symbols('t', real=True)
r=2-smp.sin(t/2) # the polar equation, where r is the radius and t is the angle.
smp.plot_parametric(r*smp.cos(t),r*smp.sin(t),(t,0, 2*smp.pi)) #plot using␣
 ↪parameteric equations
```



[37]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b88293070>
```

3.3.1 Conversion between Cartesian Coordinates and Polar Coordinates

Supppose that we want to transfrom equation from $x - y$ cartesian coordinates to polar coordinates. The starting point is simply to substitute

$$x = r\cos\theta,$$
$$y = r\sin\theta$$

We will use Python Sympy to execute this.

Example 3.7. Express $x^2 + 4x + y^2 + 6x = 0$ in polar coordinates.

We can answer this through following code

[3]:
```
import sympy as sp

# Define symbols
x, y, r, theta = sp.symbols('x y r theta')

# Original Cartesian equation
```

```
cartesian_eq = x**2 + 4*x + y**2 + 6*y

# Substitute polar coordinate relationships
polar_substitutions = {
    x: r * sp.cos(theta),
    y: r * sp.sin(theta)
}

# Convert to polar form
polar_eq = cartesian_eq.subs(polar_substitutions)

# Simplify the equation
simplified_polar_eq = sp.simplify(polar_eq)

# Display results
print("Original Cartesian equation:")
print(sp.Eq(cartesian_eq, 0))

print("\nAfter substituting polar coordinates:")
print(sp.Eq(polar_eq, 0))

print("\nSimplified polar equation:")
print(sp.Eq(simplified_polar_eq, 0))

# Optional: Solve for r
solutions = sp.solve(simplified_polar_eq, r)
print("\nSolutions for r:")
for sol in solutions:
    print(sp.Eq(r, sol))
```

```
Original Cartesian equation:
Eq(x**2 + 4*x + y**2 + 6*y, 0)

After substituting polar coordinates:
Eq(r**2*sin(theta)**2 + r**2*cos(theta)**2 + 6*r*sin(theta) + 4*r*cos(theta), 0)

Simplified polar equation:
Eq(r*(r + 6*sin(theta) + 4*cos(theta)), 0)

Solutions for r:
Eq(r, 0)
Eq(r, -6*sin(theta) - 4*cos(theta))
```

To convert from polar coordinate to cartesian coordinate, we use the following identities

$$r = \sqrt{x^2 + y^2},$$
$$\theta = \tan^{-1}\left(\frac{y}{x}\right)$$

Example 3.8. Change the polar equation $r^2 = \sin\theta$ into $x - y$ cartesian coordinate.

[10]:
```
import sympy as sp

# Define symbols
x, y, r, theta = sp.symbols('x y r theta')

# Original polar equation
polar_eq = r**2 - sp.sin(theta)

# Substitute Cartesian coordinate relationships
cartesian_substitutions = {
    r: sp.sqrt(x**2 + y**2),
    theta: sp.atan2(y, x),
    sp.sin(theta): y / sp.sqrt(x**2 + y**2)  # sin(θ) = y/r
}

# Convert to Cartesian form
cartesian_eq = polar_eq.subs(cartesian_substitutions)

# Multiply both sides by r to eliminate denominator
cartesian_eq = cartesian_eq * sp.sqrt(x**2 + y**2)

# Simplify the equation
simplified_cartesian_eq = sp.simplify(cartesian_eq)

# Display results
print("Original polar equation:")
print(sp.Eq(polar_eq, 0))

print("\nAfter substituting Cartesian coordinates:")
print(sp.Eq(cartesian_eq, 0))

print("\nSimplified Cartesian equation:")
print(sp.Eq(simplified_cartesian_eq, 0))

# Alternative form (optional)
print("\nAlternative form (x² + y²)^(3/2) = y:")
print(sp.Eq((x**2 + y**2)**(sp.Rational(3,2)),y))
```

```
Original polar equation:
Eq(r**2 - sin(theta), 0)

After substituting Cartesian coordinates:
Eq(sqrt(x**2 + y**2)*(x**2 + y**2 - y/sqrt(x**2 + y**2)), 0)

Simplified Cartesian equation:
Eq(-y + (x**2 + y**2)**(3/2), 0)
```

```
Alternative form (x² + y²)^(3/2) = y:
Eq((x**2 + y**2)**(3/2), y)
```

3.3.2 Calculus and Polar Coordinate

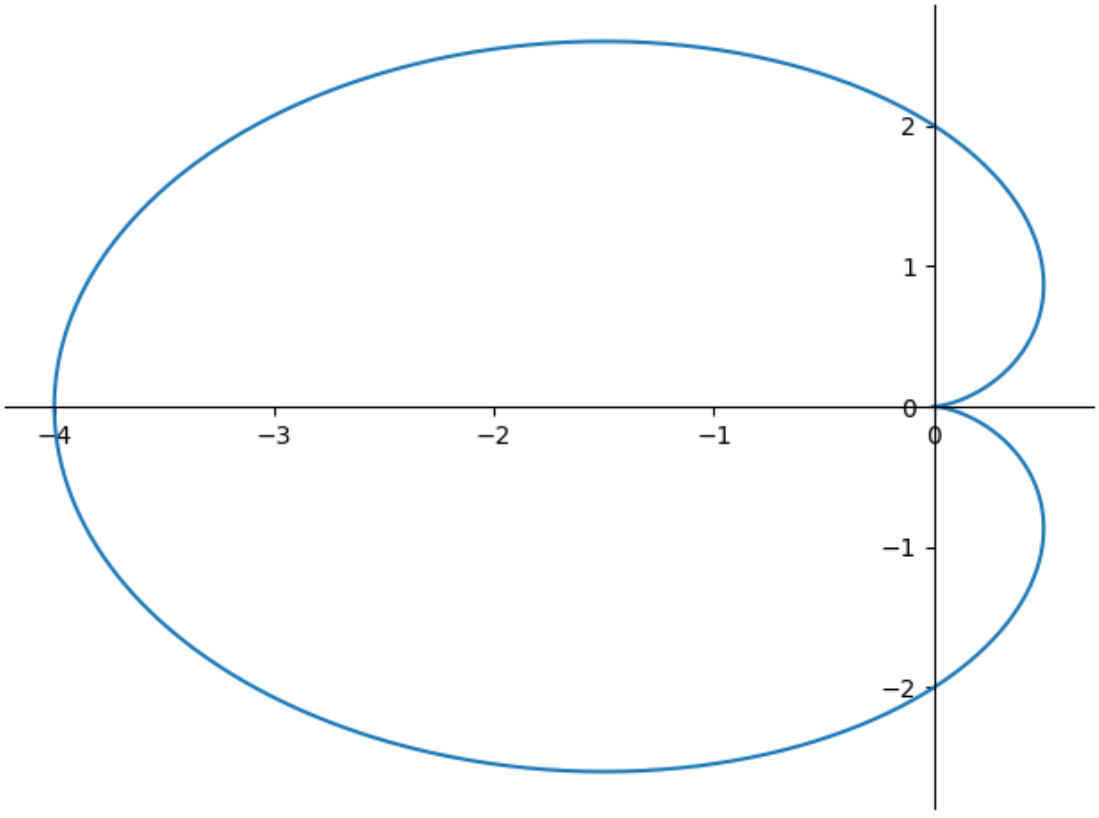
Calculus in polar coordinate can be done similar as in the parameteric equation.

Area or Arclength in Polar

Example 3.9. Find the length of the arc from $t = 0$ to $t = 2\pi$ of cardioid $r = 2 - 2\cos(t)$

To get better understanding the problem, we plot the corresponding curve

[38]:
```
t = smp.symbols('t', real=True) #Defines t as a symbolic variable representing␣
↪the angle

r = 2 - 2*smp.cos(t) # Defines the polar equation for the cardioid

smp.plot_parametric(r*smp.cos(t),r*smp.sin(t),(t,0,2*smp.pi))  # Plots the␣
↪cardioid using parametric equations
```



[38]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b9b7bee80>
```

The arclength for a polar curve is given by:

$$s = \int_a^b \sqrt{[f'(\theta)]^2 + [f(\theta)]^2} d\theta$$

[39]:
```
drdt = smp.diff(r,t) # Calculates the derivative of r with respect to t

Length = smp.integrate(smp.sqrt(r**2+drdt**2),(t,0,2*smp.pi)) #Calculates the␣
 ↪arc length using the formula for arc length in polar coordinates

print("The arclength of object is")
print(Length.evalf())
```

```
The arclength of object is
16.0000000000000
```

Chapter 4

Sequence and Series

This module explores the application of symbolic computation with Sympy to analyze the convergence, divergence, and summation of sequences and calculation of Taylor series in calculus, demonstrating key techniques for solving problems related to infinite sums.

4.1 Sequence of Real Numbers

The general procedure to solve this problem in Sympy, we follow these steps:

1. Define the variable n as positive integer
2. Define the sequence rule or formula
3. Calculate the limit of sequence as n approaches infinity. If the limit exist and is inite than the sequnce is convergent, otherwise it is divergent

Example 4.1. Determine whether the sequence defined by $a_n = \frac{2n+1}{n+1}$ converges or diverges.

The code below implements this procedure:

[40]:
```
import sympy as smp
n = smp.Symbol("n", integer = True, positive = True) # defines 'n' as a␣
 ↪positive integer
an = (2*n+1)/(n+1) # the sequence's general term
L = smp.limit(an,n,smp.oo) # calculates the limit
L
```

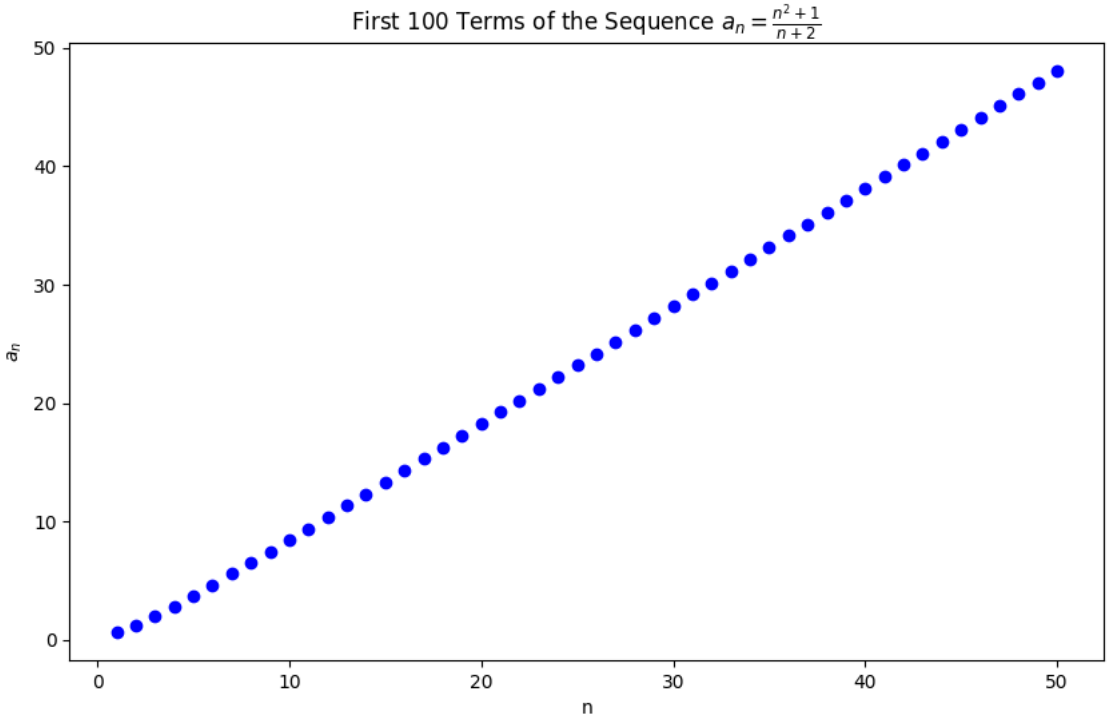
[40]:
$$2$$

Since the limit exists and is finite, we can conclude that the sequence is convergent.

Example 4.2. Plot the the first 50 terms of the sequence $a_n = \frac{n^2+1}{n+2}$

[41]:
```
import sympy as smp
import matplotlib.pyplot as plt


n = smp.symbols('n') # Define the variable n
```

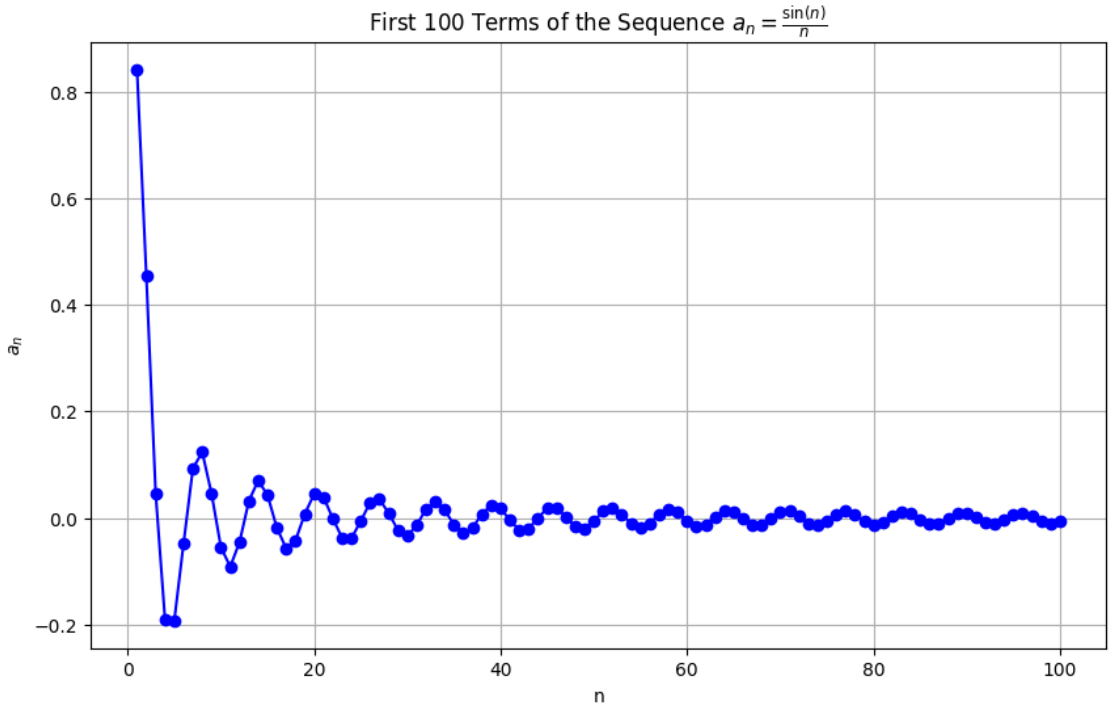
```
a_n = (n**2 + 1) / (n + 2)# Define the sequence
terms = [a_n.subs(n, i).evalf() for i in range(1, 51)] # Compute the first 50␣
 ↪terms of the sequence

# Plot the first 100 terms of the sequence
plt.figure(figsize=(10, 6))
plt.scatter(range(1, 51), terms, marker='o', color='b')
plt.title('First 100 Terms of the Sequence $a_n = \\frac{n^2 + 1}{n + 2}$')
plt.xlabel('n')
plt.ylabel('$a_n$')
plt.show()
```



Example 3: Determine if the sequence $a_n = \frac{\cos(n)}{n}$ is increasing, decreasing, or neither. Use SymPy to perform the calculations and plot the first 100 terms of the sequence to verify your result.

[42]:
```
import sympy as smp
import matplotlib.pyplot as plt

# Define the variable n
n = smp.symbols('n')

# Define the sequence
a_n = smp.sin(n) / n
```

```
# Compute the first 100 terms of the sequence
terms = [a_n.subs(n, i).evalf() for i in range(1, 101)]


# Plot the first 100 terms of the sequence
plt.figure(figsize=(10, 6))
plt.plot(range(1, 101), terms, marker='o', linestyle='-', color='b')
plt.title('First 100 Terms of the Sequence $a_n = \\frac{\\sin(n)}{n}$')
plt.xlabel('n')
plt.ylabel('$a_n$')
plt.grid(True)
plt.show()
```

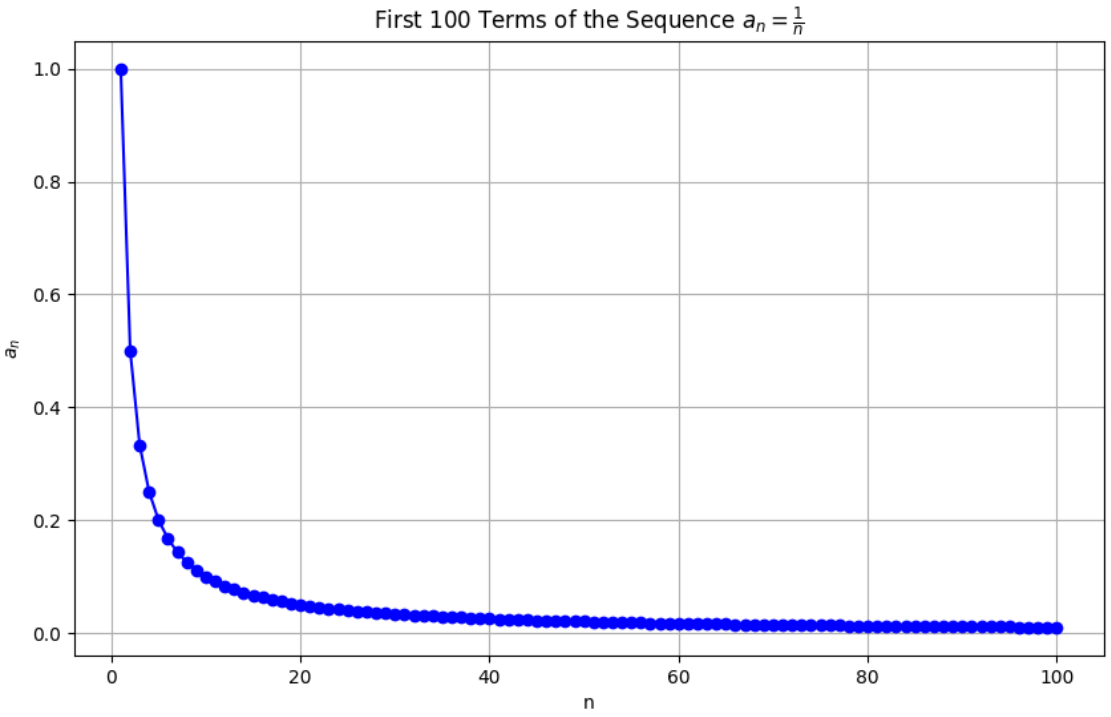


from the figure we can observe that the series is oscillating, meaning it is neither increasing or decreasing.

Example 4.3. Determine if the sequence $a_n = \frac{1}{n}$ is increasing, decreasing, or neither. To get general idea, we plot the first 100 terms of the sequence.

[43]:
```
import sympy as smp
import matplotlib.pyplot as plt

# Define the variable n
n = smp.symbols('n', integer=True, positive=True)
```

```
# Define the sequence
a_n = 1 / n
# Compute the first 100 terms of the sequence
terms = [a_n.subs(n, i).evalf() for i in range(1, 101)]

# Plot the first 100 terms of the sequence
plt.figure(figsize=(10, 6))
plt.plot(range(1, 101), terms, marker='o', linestyle='-', color='b')
plt.title('First 100 Terms of the Sequence $a_n = \\frac{1}{n}$')
plt.xlabel('n')
plt.ylabel('$a_n$')
plt.grid(True)
plt.show()
```



We might suspect that it is always decreasing, to check we find whether the difference between terms is always positive, always negative or neither:

[44]:
```
# Compute the difference a_(n+1) - a_n
difference = a_n.subs(n, n + 1) - a_n

# Simplify the difference
difference_simplified = smp.simplify(difference)
```

```
# Determine the monotonicity analytically using solve_univariate_inequality
increasing_inequality = smp.solve_univariate_inequality(difference_simplified >␣
↪0, n, relational=False)
decreasing_inequality = smp.solve_univariate_inequality(difference_simplified <␣
↪0, n, relational=False)

# Print the result
if increasing_inequality:
    monotonicity = "increasing"
elif decreasing_inequality:
    monotonicity = "decreasing"
else:
    monotonicity = "neither increasing nor decreasing"

print(f"The sequence is {monotonicity}.")
```

```
The sequence is decreasing.
```

4.2 Infinite Series

Example 4.4. Calculate the sum of the infinite series

$$\sum_{k=1}^{\infty} \frac{9}{k(k+3)}$$

To calculate the sum of the series, we first perform partial fraction decomposition on the general term using Sympy's apart function:

[45]:
```
import sympy as smp
k = smp.Symbol("k", positive = True) # we do not use integer = True as the␣
↪partial fraction is not supported for integer
ak = 9/(k*(k+1)) # Defines the expression an
ak1 = smp.apart(ak) #Calculates the partial fraction decomposition of an
ak1
```

[45]:
$$-\frac{9}{k+1}+\frac{9}{k}$$

Next, we calculate the partial sums sums of the series:

[46]:
```
n = smp.Symbol("n", integer = True, positive = True)
Sn = smp.Sum(ak1,(k,1,n)).doit() # calculate the partial sum until n
Sn
```

[46]:
$$9-\frac{9}{n+1}$$

Finally, we evaluate the limit of the partial sums as $n \to \infty$

[47]:
```
smp.limit(Sn,n,smp.oo) # compute the series limit using its partial sum
```

[47]:

$$9$$

Hence, the infinite series converges to a sum of 9

4.3 Integral Test

Example 4.5. Use the integral test to determine if the series

$$\sum_{n=1}^{\infty} \frac{1}{\sqrt[3]{n}}$$

converges or diverges.

To apply the integral test, the function $f(x) = \frac{1}{\sqrt[3]{x}}$, must be positive, continuous and decreasing on the interval $[1, \infty)$. We will verify these conditions using Sympy. First, we usesolve_univariate_inequality from `sympy.solvers.inequalities` to find the values of x for which f is positive. Then, we use `continous_domain` from `sympy.calculus.util` to find the domain where the function f is continuous.

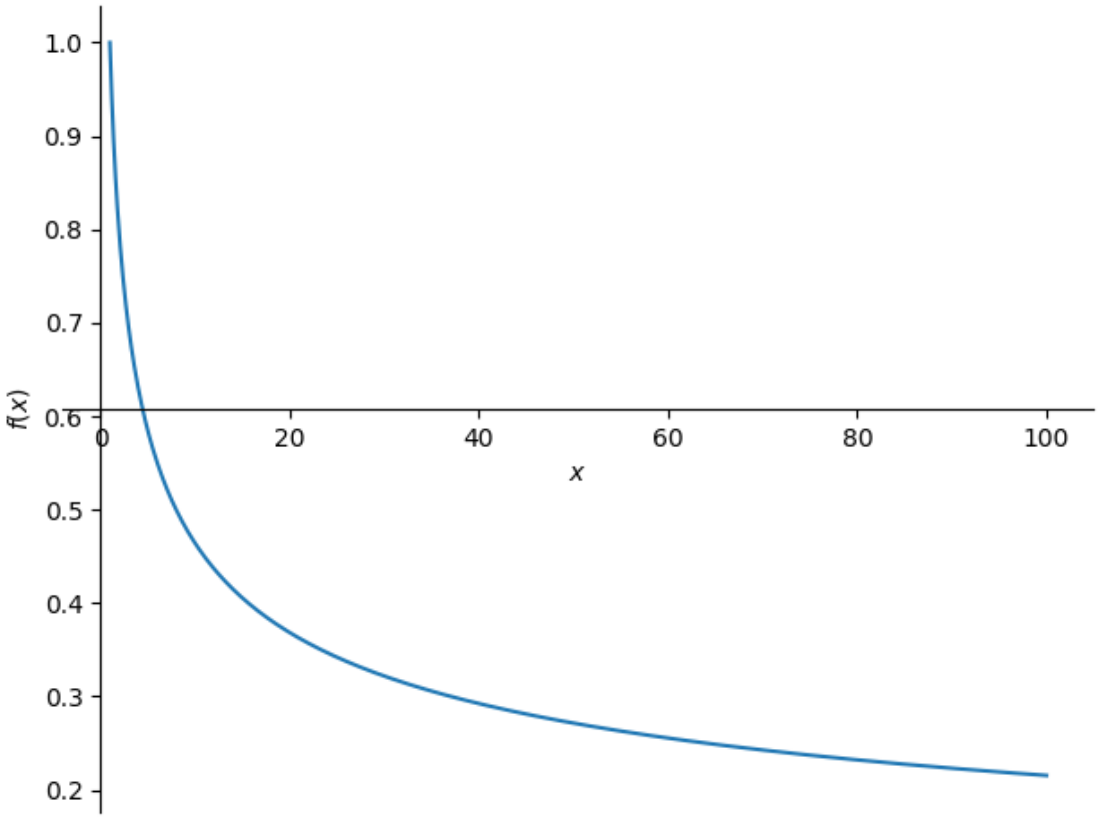
[48]:
```
from sympy.solvers.inequalities import solve_univariate_inequality
from sympy.calculus.util import continuous_domain
x = smp.Symbol("x",real = True) # Define x as a real-valued symbol
f = 1/smp.root(x,3) # the function
f
```

[48]:

$$\frac{1}{\sqrt[3]{x}}$$

We can plot the function to help visualize this condition:

[49]:
```
smp.plot(f,(x,1,100))
```



[49]:
```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at
0x18b99bc6730>
```

We check the positivity conditon:

[50]:
```
solve_univariate_inequality(f>=0,x) # Solve the inequality f(x) >= 0
```

[50]:
$0 < x$

This means that the function is positive for all $x > 0$, which includes the interval $[1, \infty)$.

Next, we determine the domain where the function is continous using the following code:

[51]:
```
continuous_domain(f, x, smp.Reals) # Find the continuous domain of f(x)
```

[51]:
$(-\infty, 0) \cup (0, \infty)$

The output means that f is continous for all positive real number except at $x = 0$. Therefore, the interval $[1, \infty)$ is included as well.

To determine where the function is decreasing, we need to find where the derivative of f is negative. This can be done by once again using `solve_univariate_inequality`

[52]:
```
df=smp.diff(f,x) # calculate the derivative of f
solve_univariate_inequality(df < 0, x) # find the domain which f is decreasing
```

[52]:
$0 < x$

This result indicates that the derivative is negative for all $(0 < x)$, including the interval $[1, \infty)$.

Since all the conditions are satisfied, we can use the integral test on the series:

[53]:
```
smp.integrate(f,(x,1,smp.oo)) # Evaluate the improper integral of f(x) from 1␣
↪to infinity (for the Integral Test)
```

[53]:
∞

Since the integral diverges, we can conclude that the series also diverges by the integral test.

4.4 Limit Comparison Test

Example 4.6. Determine whether the series

$$\sum_{n=1}^{\infty} \frac{1}{n^2+3}$$

converges or diverges using the limit comparison test.

To apply the limit comparison test, we compare the given series with the series $\sum_{n=1}^{\infty} \frac{1}{n^2}$. Since both series have positive terms, the limit comparison test is applicable. We use $a_n = \frac{1}{n^2+3}$ while $b_n = \frac{1}{n^2}$

[54]:
```
n = smp.Symbol("n", integer = True, positive = True) # Define n as a positive␣
↪integer
an = 1/(n**2+3) # Define the general term of the given series
bn = 1/(n**2) # Define the general term of the comparison series
R = an/bn # Calculate the ratio of the terms
R
```

[54]:
$$\frac{n^2}{n^2+3}$$

Next, we evaluate the limit of the ratio $R = \frac{a_n}{b_n}$as n approaches infinity:

[55]:
```
L = smp.limit(R,n,smp.oo) # Calculate the limit of the ratio as n approaches␣
↪infinity
L
```

[55]:
1

Since the limit of the ratio is finite and positive, and we know that $\sum_{n=1}^{\infty} \frac{1}{n^2}$ is converge (it is a p-series with $p = 2$), we conclude by the limit comparison test that $\sum_{n=1}^{\infty} \frac{1}{n^2+3}$ is a convergent series

4.5 Alternating series

Example 4.7. Determine whether the series is converge or diverge

$$\sum_{n=1}^{\infty}(-1)^n\frac{4}{\sqrt{n}}$$

This is an alternating series. To determine convergence, we need to verify that the sequence of absolute values of the terms, $a_n = \frac{4}{\sqrt{n}}$, satisfies the conditions $0 < a_{n+1} < a_n$ for all $n \geq 1$ and $\lim_{n\to\infty} a_n = 0$. We can begin by checking the first inequality, $0 < a_{n+1}$, using the following code:"

[56]:
```
from sympy.solvers.inequalities import solve_univariate_inequality
n = smp.Symbol("n", integer = True, positive = True)  # Define n as a positive integer
an = 4/(smp.sqrt(n)) # Define the general term of the given series
```

[57]:
```
solve_univariate_inequality(an > 0, n) # find the domain which an is positive
```

[57]:
True

meaning that the series is always positive for all positive integer n.

To verify the second condition, $a_{n+1} < a_n$, we can consider two approaches. First, we can directly compare the terms a_{n+1} and a_n by examining their difference.

[58]:
```
an1 = an.subs(n,n+1)  # the n+1 term
D = an1-an #the difference between successive term
D
```

[58]:
$$\frac{4}{\sqrt{n+1}} - \frac{4}{\sqrt{n}}$$

[59]:
```
solve_univariate_inequality(D < 0, n) # find the domain which the difference is positive
```

[59]:
True

which means that $a_{n+1} < a_n$ for all $n > 0$.

Alternatively, we can analyze the continuous analog of the sequence, $f(x) = \frac{4}{\sqrt{x}}$, and check if its derivative is negative. A negative derivative would indicate that the function, and therefore the sequence, is monotonically decreasing."

[60]:
```
x = smp.Symbol("x",real = True) # Define x as a real-valued symbol
f = 4/(smp.sqrt(x))  # Define the continuous analog of the sequence's terms
df = smp.diff(f,x) # Calculate the derivative of f
df
```

[60]:
$$-\frac{2}{x^{\frac{3}{2}}}$$

[61]:
```
solve_univariate_inequality(df < 0, x) # Solve the inequality df < 0
```

[61]:
$$0 < x$$

since f is always decreasing for $x > 0$, we can conclude that which means that $a_{n+1} < a_n$ for all $n > 0$.

For the final condition, we will show that $\lim_{n\to\infty} a_n = 0$, which can be done by taking the limit of a_n:

[62]:
```
L = smp.limit(an,n,smp.oo) # Calculate the limit of the series as n approaches␣
↪infinity
L
```

[62]:
$$0$$

Therefore, as all the hypotheses of the Alternating Series Test are satisfied, the series converges.

4.6 Ratio Test

Example 4.8. Determine whether the series

$$\sum_{n=1}^{\infty} \frac{3^n}{n!}$$

converges or diverges using the Ratio Test.

To apply the Ratio Test, we need to evaluate the limit

$$L = \lim_{n\to\infty} \left| \frac{a_{n+1}}{a_n} \right|.$$

We begin by calculating the ratio $\left|\frac{a_{n+1}}{a_n}\right|$:

[63]:
```
n = smp.Symbol("n", integer = True, positive = True) # Define n as a positive␣
↪integer
an = 3**n/smp.factorial(n) # Define the general term of the series
an1 = an.subs(n,n+1)
R = abs(an1/an)
R
```

[63]:
$$\frac{3^{-n}3^{n+1}n!}{(n+1)!}$$

Before evaluating the limit, it is often beneficial to simplify the expression

[64]:
```
bn = R.simplify() #simplifying the expression
bn
```

[64]:
$$\frac{3}{n+1}$$

Finally, we calculate the limit:

[65]:
```
smp.limit(bn,n,smp.oo) # calculating the limit
```

[65]:
$$0$$

Since the limit of the ratio is 0, which is less than 1, we can conclude by the Ratio Test that the series converges.

Example 4.9. Apply the Root Test to determine if the series $\sum_{n=1}^{\infty} \left(\frac{2^n}{n^2}\right)^n$ converges or diverges.

[66]:
```
import sympy as smp

# Define the variable n
n = smp.symbols('n', integer=True, positive=True)

# Define the general term of the series
a_n = (2**n / n**2)**n

# Apply the Root Test
root_test = smp.limit(smp.Abs(a_n)**(1/n), n, smp.oo)

# Determine convergence or divergence
if root_test < 1:
    result = "converges"
elif root_test > 1:
    result = "diverges"
else:
    result = "inconclusive"

result
```

[66]:
```
'diverges'
```

4.7 Taylor Series

Example 4.10. Use the first five terms of the Taylor series of $f(x) = \frac{1}{1-x}$ around $x = 0$ to approximate the definite integral $\int_0^{0.1} f(x)dx$

First, we use Sympy to calculate the first five terms of the Taylor series

[67]:
```
import sympy as smp

x = smp.Symbol('x') # Define x as a symbolic variable
f = 1/(1-x) # Define the function

taylor_series = f.series(x, x0=0, n=5)  # Calculate Taylor series around x=0 up␣
 ↪to order 5

print(taylor_series)
```

```
1 + x + x**2 + x**3 + x**4 + O(x**5)
```

To remove the $O(x^5)$ term (representing the remainder), we can use the removeO() method:

[68]:
```
taylor_series_trun = taylor_series.removeO()  # Remove the O(x^5) term
print(taylor_series_trun)
```

```
x**4 + x**3 + x**2 + x + 1
```

Now, we can approximate the definite integral using the truncated Taylor series:

[69]:
```
smp.integrate(taylor_series_trun,(x,-0.1,0.1)) # calculating the integral using␣
↪truncated Taylor series
```

[69]:
$$0.200670666666667$$

Chapter 5

Vectors and the Geometry of Space

In this tutorial, we will explore how to solve vector problems and understand the geometry of space using Python, leveraging both NumPy and SymPy. Vectors are essential mathematical objects used to represent quantities with both magnitude and direction. They play a critical role in fields such as physics, engineering, computer graphics, and data analysis.

In this section, we will cover key topics such as:

- Vector basics: Definition, vector addition, subtraction, and scalar multiplication.
- Dot product: Calculating the angle between two vectors and understanding vector projection.
- Cross product: Determining a vector perpendicular to two given vectors in 3D space.
- Magnitude and unit vectors: Measuring vector length and normalizing vectors.
- Geometric applications: Vector projections, distances between points, and angles in space.
- Symbolic operations: Using SymPy for symbolic vector expressions, simplifications, and calculus.

By the end of this tutorial, you'll be able to use Python to solve vector-related problems, perform symbolic computations, and visualize geometric relationships in space, equipping you with valuable tools for applications in science and engineering.

5.1 Visualizing Vectors

Example 5.1. Plot the following 3D vectors: $\mathbf{V_1} == (2, 3)$ and $\mathbf{V_2} = (2, 3)$

A vector has direction. In this case: - $\mathbf{V_1}$ direction: 2 to the right, 3 up - $\mathbf{V_2}$ direction: 1 to the left, 2 up

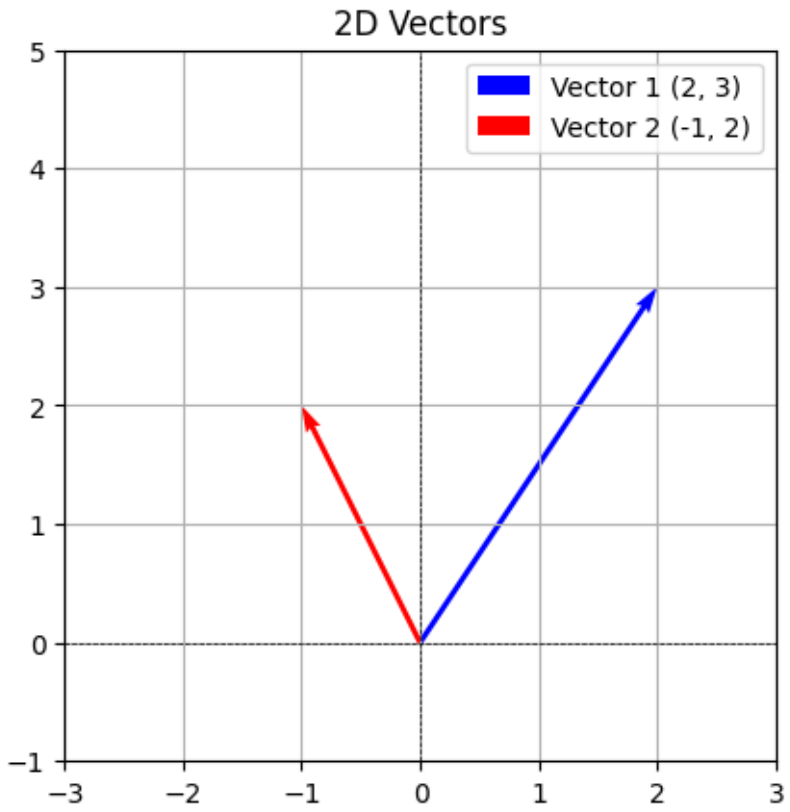
In 2D, usually the direction are direction - right = x positive - left = x negative - up = y positive - down = y negative

[70]:
```
# Make sure the library are installed, if not, uncomment the following lines to␣
↪install library
# pip install sympy
# pip install numpy
```

```
# pip install matplotlib
```

[71]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

[72]:
```
# Define the origin and vectors
initial_point = np.array([0, 0])  # Initial point
vector1 = np.array([2, 3])  # Vector 1
vector2 = np.array([-1, 2])  # Vector 2

# Create the figure and axis
plt.figure()
# symbols * is used to unpack the array into individual elements
plt.quiver(*initial_point, *vector1, color='b', angles='xy', scale_units='xy',␣
 ↪scale=1, label='Vector 1 (2, 3)')
plt.quiver(*initial_point, *vector2, color='r', angles='xy', scale_units='xy',␣
 ↪scale=1, label='Vector 2 (-1, 2)')

# Set limits and grid
plt.xlim(-3, 3)
plt.ylim(-1, 5)
plt.axhline(0, color='black',linewidth=0.5, ls='--')
plt.axvline(0, color='black',linewidth=0.5, ls='--')
plt.grid()
plt.title('2D Vectors')
plt.legend()
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```



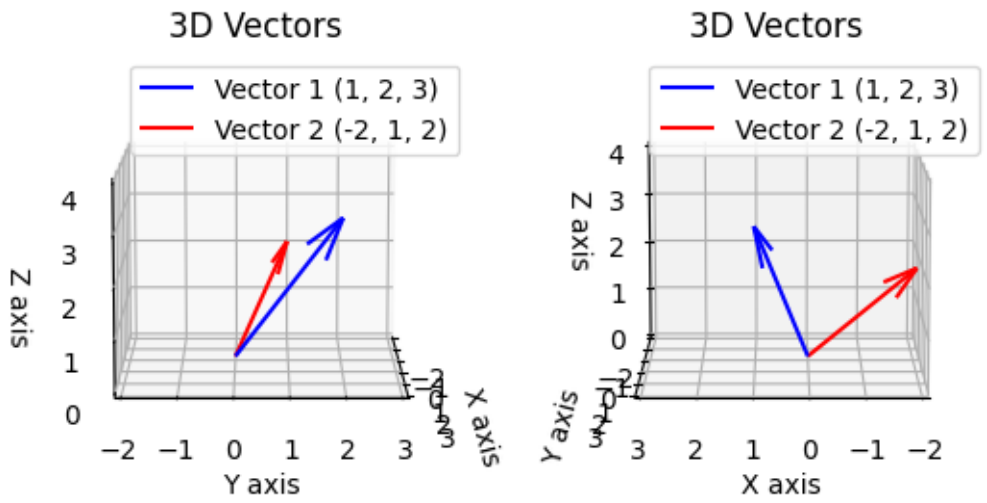
5.2 Vectors in Space

Example 5.2. Plot the following 3D vectors: $\mathbf{V_1} = (1, 2, 3)$ and $\mathbf{V_2} = (-2, 1, 2)$

A vector has direction. In this case $\mathbf{V_1}$ direction: 1 to x positive, 2 to y positive, 3 to z positive while $\mathbf{V_2}$ direction: -2 to x negaive, 1 to y positive, 2 to z positive

[73]:
```
# Create a 3D plot
fig = plt.figure()
ax = fig.add_subplot(121, projection='3d')

# Define the origin and vectors
origin = np.array([0, 0, 0])  # Origin point
vector1 = np.array([1, 2, 3])  # Vector 1
vector2 = np.array([-2, 1, 2])  # Vector 2

# Plot the vectors
# symbols * is used to unpack the array into individual elements
ax.quiver(*origin, *vector1, color='b', label='Vector 1 (1, 2, 3)',␣
 ↪arrow_length_ratio=0.25)
```

```
ax.quiver(*origin, *vector2, color='r', label='Vector 2 (-2, 1, 2)',␣
 ↪arrow_length_ratio=0.25)

# Set limits
ax.set_xlim([-2, 3])
ax.set_ylim([-2, 3])
ax.set_zlim([0, 4])

# Labels and title
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_zlabel('Z axis')
ax.set_title('3D Vectors')
ax.legend()
ax.view_init(elev=10, azim=0)

ax2 = fig.add_subplot(122, projection='3d')

ax2.quiver(*origin, *vector1, color='b', label='Vector 1 (1, 2, 3)',␣
 ↪arrow_length_ratio=0.25)
ax2.quiver(*origin, *vector2, color='r', label='Vector 2 (-2, 1, 2)',␣
 ↪arrow_length_ratio=0.25)

# Set limits
ax2.set_xlim([-2, 3])
ax2.set_ylim([-2, 3])
ax2.set_zlim([0, 4])

# Labels and title
ax2.set_xlabel('X axis')
ax2.set_ylabel('Y axis')
ax2.set_zlabel('Z axis')
ax2.set_title('3D Vectors')
ax2.legend()
ax2.view_init(elev=10, azim=90)
plt.show()
```



To better understand vector operations and their geometric meaning, especially in 3D, it's important to visualize them. > Imagination helps bridge the gap between abstract math and practical, spatial understanding.

5.3 Dot Product

Given two vectors $\mathbf{a}, \mathbf{b}$ below are the methods to calculate them.

Step 1: Calculate the dot product $mathbfa \cdot \mathbf{b}$

Step 2: Calculate the magnitudes $\|\mathbf{a}\|\|\mathbf{b}\|$

Step 3: Compute $\cos(\theta) = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$

Step 4: Calculate the angle θ

This gives you the angle between the two vectors $\mathbf{a}$ and $\mathbf{b}$.

Understanding how to calculate the angle between two vectors is essential for applications in physics, computer graphics, and various engineering disciplines.

Example 5.3. Determine the angle between the vectors $\mathbf{a} = \begin{pmatrix} 0 \\ 3 \\ 4 \end{pmatrix}$ and $\mathbf{b} = \begin{pmatrix} 5 \\ 0 \\ -12 \end{pmatrix}$.

[74]:
```
# We use the SymPy library for symbolic mathematics, which allows us to perform␣
↪algebraic computations.
# If we want numerical (decimal) results, we would use the NumPy library␣
↪instead.

import sympy as sp  # Import the SymPy library and give it the alias 'sp'.

# Define the vectors using SymPy's Matrix function
```

```
a = sp.Matrix([0, 3, 4])  # Vector 'a' represented as a column matrix.
b = sp.Matrix([5, 0, -12])  # Vector 'b' represented as a column matrix.

# Calculate the magnitudes (lengths) of the vectors
magnitude_a = a.norm()  # Compute the magnitude of vector 'a'.
magnitude_b = b.norm()  # Compute the magnitude of vector 'b'.

# Calculate the dot product of the two vectors
dot_product = a.dot(b)  # Find the dot product of vector 'a' and vector 'b'.

# Calculate the cosine of the angle between the two vectors using the dot␣
 ↪product and their magnitudes
cos_theta = dot_product / (magnitude_a * magnitude_b)  # Cosine of the angle =␣
 ↪dot product / (magnitude of a * magnitude of b)

# Calculate the angle in radians using the arccosine function
angle_radians = sp.acos(cos_theta)  # Calculate the angle in radians from the␣
 ↪cosine value.

# Convert the angle from radians to degrees for easier interpretation
angle_degrees = sp.deg(angle_radians)  # Convert the angle in radians to␣
 ↪degrees.

# Display the results in the output
print(f"Vector a= {a}")  # Print vector 'a'.
display(a)  # Use the display function to show the vector 'a' nicely.

print(f"Vector b= {b}")  # Print vector 'b'.
display(b)  # Use the display function to show the vector 'b' nicely.

print(f"Length/Magnitude/Norm of Vector a = ||a||= {magnitude_a}")  # Print the␣
 ↪magnitude of vector 'a'.
display(magnitude_a)  # Display the magnitude of vector 'a'.

print(f"Length/Magnitude/Norm of Vector b = ||b||= {magnitude_b}")  # Print the␣
 ↪magnitude of vector 'b'.
display(magnitude_b)  # Display the magnitude of vector 'b'.

# print(f"Addition of Vector a + b = {a + b}")  # Print the result of adding␣
 ↪vector 'a' and vector 'b'.
# display(a + b)  # Display the result of the addition.

# print(f"Subtraction of Vector a - b = {a - b}")  # Print the result of␣
 ↪subtracting vector 'b' from vector 'a'.
# display(a - b)  # Display the result of the subtraction.
```

```
# print(f"Scalar Multiplication of Vector a * 2 = {a * 2}")  # Print the result␣
 ↪of multiplying vector 'a' by 2.
# display(a * 2)  # Display the result of the scalar multiplication.

print(f"Unit Vector a = {a / magnitude_a}")  # Print the unit vector of 'a',␣
 ↪which has a magnitude of 1.
display(a / magnitude_a)  # Display the unit vector of 'a'.

print(f"Unit Vector b = {b / magnitude_b}")  # Print the unit vector of 'b',␣
 ↪which has a magnitude of 1.
display(b / magnitude_b)  # Display the unit vector of 'b'.

print(f"Dot Product of Vector a . b = {dot_product}")  # Print the dot product␣
 ↪of vector 'a' and vector 'b'.
display(dot_product)  # Display the dot product.

print(f"Cosine of angle between vector a and b = {cos_theta} = {cos_theta.
 ↪evalf()}")  # Print the cosine of the angle between vectors 'a' and 'b'.
display(cos_theta)  # Display the cosine value.

print(f"Angle of vector a and b in radians = {angle_radians} = {angle_radians.
 ↪evalf()}")  # Print the angle in radians.
display(angle_radians)  # Display the angle in radians.

print(f"Angle of vector a and b in degrees = {angle_degrees} = {angle_degrees.
 ↪evalf()}")  # Print the angle in degrees.
display(angle_degrees)  # Display the angle in degrees.
```

```
Vector a= Matrix([[0], [3], [4]])
```

$$\begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}$$

```
Vector b= Matrix([[5], [0], [-12]])
```

$$\begin{bmatrix} 5 \\ 0 \\ -12 \end{bmatrix}$$

```
Length/Magnitude/Norm of Vector a = ||a||= 5
```

$$5$$

```
Length/Magnitude/Norm of Vector b = ||b||= 13
```

$$13$$

```
Unit Vector a = Matrix([[0], [3/5], [4/5]])
```

$$\begin{bmatrix} 0 \\ \frac{3}{5} \\ \frac{4}{5} \end{bmatrix}$$

```
Unit Vector b = Matrix([[5/13], [0], [-12/13]])
```

$$\begin{bmatrix} \frac{5}{13} \\ 0 \\ -\frac{12}{13} \end{bmatrix}$$

```
Dot Product of Vector a . b = -48
```

$$-48$$

```
Cosine of angle between vector a and b = -48/65 = -0.738461538461539
```

$$-\frac{48}{65}$$

```
Angle of vector a and b in radians = acos(-48/65) = 2.40158224226860
```

$$\operatorname{acos}\left(-\frac{48}{65}\right)$$

```
Angle of vector a and b in degrees = 180*acos(-48/65)/pi = 137.600526635555
```

$$\frac{180 \operatorname{acos}\left(-\frac{48}{65}\right)}{\pi}$$

5.4 Cross Product and Area of a Parallelogram

Example 5.4. Determine the cross product of the vectors $\mathbf{a} = \begin{pmatrix} 0 \\ 3 \\ 4 \end{pmatrix}$ and $\mathbf{b} = \begin{pmatrix} 5 \\ 0 \\ -12 \end{pmatrix}$. Calculate the area of the parallelogram formed by these vectors using the cross product and the magnitudes of the vectors.

[75]:
```
# Define the vectors using SymPy's Matrix function
a = sp.Matrix([0, 3, 4])  # Define vector 'a' as a column matrix.
b = sp.Matrix([5, 0, -12])  # Define vector 'b' as a column matrix.

# Calculate the cross product of the two vectors
cross_product = a.cross(b)  # Compute the cross product of vectors 'a' and 'b'.

# Calculate the area of the parallelogram using the norm (magnitude) of the␣
↪cross product
area_by_norm_of_cross = cross_product.norm()  # The area of the parallelogram␣
↪is equal to the magnitude of the cross product.

# alternatively, we can calculate the area using the magnitudes of the vectors␣
↪and the sine of the angle between them
# Calculate the dot product of the two vectors
```

```
dot_product = a.dot(b)  # Find the dot product of vector 'a' and vector 'b'.

# Calculate the cosine of the angle between the two vectors
cos_theta = dot_product / (magnitude_a * magnitude_b)  # Use the dot product to␣
↪calculate the cosine of the angle.

# Calculate the angle in radians
angle_radians = sp.acos(cos_theta)  # Calculate the angle in radians from the␣
↪cosine value.

# Calculate the area of the parallelogram using the sine of the angle
# The area can also be computed as the product of the magnitudes of the vectors␣
↪and the sine of the angle between them
area_by_norm_of_magnitude_sin = a.norm() * b.norm() * sp.sin(angle_radians)  #␣
↪Area = |a| * |b| * sin(theta)

# Display the results in the output
print(f"Vector a= {a}")  # Print vector 'a'.
display(a)  # Display vector 'a' nicely.

print(f"Vector b= {b}")  # Print vector 'b'.
display(b)  # Display vector 'b' nicely.

print(f"Cross Product (a x b)= {cross_product}")  # Print the cross product of␣
↪vectors 'a' and 'b'.
display(cross_product)  # Display the cross product.

print(f"Area of the Parallelogram (using cross product)=␣
↪{area_by_norm_of_cross} = {area_by_norm_of_cross.evalf()}")  # Print the␣
↪area calculated using the norm of the cross product.
display(area_by_norm_of_cross)  # Display the area from the cross product␣
↪calculation.

print(f"Area of the Parallelogram (using sine)=␣
↪{area_by_norm_of_magnitude_sin}= {area_by_norm_of_magnitude_sin.evalf()}") ␣
↪# Print the area calculated using the magnitudes and sine of the angle.
display(area_by_norm_of_magnitude_sin)  # Display the area from the sine␣
↪calculation.
```

```
Vector a= Matrix([[0], [3], [4]])
```

$$\begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}$$

```
Vector b= Matrix([[5], [0], [-12]])
```

$$\begin{bmatrix} 5 \\ 0 \\ -12 \end{bmatrix}$$

```
Cross Product (a x b)= Matrix([[-36], [20], [-15]])
```

$$\begin{bmatrix} -36 \\ 20 \\ -15 \end{bmatrix}$$

```
Area of the Parallelogram (using cross product)= sqrt(1921) = 43.8292140016222
```

$$\sqrt{1921}$$

```
Area of the Parallelogram (using sine)= sqrt(1921)= 43.8292140016222
```

$$\sqrt{1921}$$

5.5 Lines and Planes in Space

5.5.1 Parametric Equation

Example 5.5. Consider a line in 3D space that passes through the point $P_0 = (1, 2, 3)$ and has a direction vector $\mathbf{d} = (4, 5, 6)$. Determine the parametric equations of the line.

[76]:
```
# Import the SymPy library for symbolic mathematics
import sympy as sp

# Define a point and a direction vector
point = sp.Matrix([1, 2, 3])  # This is a point P0 in 3D space with coordinates
 ↪(1, 2, 3)
direction = sp.Matrix([4, 5, 6])  # This is the direction vector d, which
 ↪indicates the direction of the line

# Define the parameter t
t = sp.symbols('t')  # 't' is a symbol that will be used to represent a
 ↪parameter for the line

# Parametric equations of the line
# These equations describe how the coordinates (x, y, z) change as the
 ↪parameter t varies
x = point[0] + direction[0] * t  # x-coordinate: starts at point's x-coordinate
 ↪and adds direction x-component scaled by t
y = point[1] + direction[1] * t  # y-coordinate: starts at point's y-coordinate
 ↪and adds direction y-component scaled by t
z = point[2] + direction[2] * t  # z-coordinate: starts at point's z-coordinate
 ↪and adds direction z-component scaled by t

# Display the parametric equations
```

```
print(f"Parametric equations of the line:")  # Print a message indicating the␣
 ↪following lines are the parametric equations
print(f"x(t) = {x}")  # Print the equation for x in terms of t
print(f"y(t) = {y}")  # Print the equation for y in terms of t
print(f"z(t) = {z}")  # Print the equation for z in terms of t
```

```
Parametric equations of the line:
x(t) = 4*t + 1
y(t) = 5*t + 2
z(t) = 6*t + 3
```

5.5.2 Plane Equation Given a Point and Direction

In three-dimensional space, a plane can be defined using a point and a normal vector (direction). The equation of a plane can be expressed in various forms, but one common way to represent it is through the point-normal form.

The equation of a plane can be written as:

$$a(x - x_0) + b(y - y_0) + c(z - z_0) = 0$$

Where: - (x, y, z) are the coordinates of any point P on the plane. - (x_0, y_0, z_0) are the coordinates of the given point P_0 on the plane. - (a, b, c) are the components of the normal vector $\mathbf{n}$.

Example 5.6. Find the equation of plane passing point $P_0(1, 2, 3)$ and a normal vector $\mathbf{n} = (4, -5, 6)$.

Using the point-normal form, the equation of the plane can be derived as follows:

- Substitute $x_0 = 1$, $y_0 = 2$, $z_0 = 3$, $a = 4$, $b = -5$, $c = 6$:

$$4(x - 1) - 5(y - 2) + 6(z - 3) = 0$$
$$4x - 5y + 6z - 12 = 0$$

Expanding this gives the final equation of the plane.

[77]:
```
# Import the SymPy library for symbolic mathematics
import sympy as sp

# Define the point and direction vector
point = sp.Matrix([1, 2, 3])  # This defines a point P with coordinates (1, 2,␣
 ↪3) in 3D space
direction_vector = sp.Matrix([4, -5, 6])  # This defines the direction vector d␣
 ↪with components (4, -5, 6)

# Create symbols for x, y, z
x, y, z = sp.symbols('x y z')  # These are symbolic variables that will␣
 ↪represent the coordinates of points in the plane

# The normal vector of the plane is the direction vector
```

```
normal_vector = direction_vector  # The normal vector of the plane is taken to␣
↪be the same as the direction vector
print('normal_vector =', normal_vector)  # Print the normal vector for␣
↪verification

# Plane equation: n_x * (x - x_0) + n_y * (y - y_0) + n_z * (z - z_0) = 0
# This is the formula for a plane where (n_x, n_y, n_z) is the normal vector␣
↪and (x_0, y_0, z_0) is a point on the plane
plane_eq = normal_vector[0] * (x - point[0]) + normal_vector[1] * (y -␣
↪point[1]) + normal_vector[2] * (z - point[2])

# Display the equation of the plane
plane_eq_simplified = sp.simplify(plane_eq)  # Simplify the plane equation for␣
↪a cleaner output
print(f"Equation of the plane: {plane_eq_simplified} = 0")  # Print the␣
↪equation of the plane in standard form
```

```
normal_vector = Matrix([[4], [-5], [6]])
Equation of the plane: 4*x - 5*y + 6*z - 12 = 0
```

5.5.3 Plane Equation Given Three Points

In three-dimensional space, a plane can be uniquely defined by three non-collinear points. The equation of the plane can be derived using the coordinates of these points.

Let the three points be: $P_1(x_1, y_1, z_1), P_2(x_2, y_2, z_2), P_3(x_3, y_3, z_3)$. To find the equation of the plane, we first need to determine a normal vector $\mathbf{n}$ to the plane. This can be done by taking the cross product of two vectors that lie on the plane.

1. **Calculate Vectors**:
   - Create two vectors from the three points:
     - $\mathbf{v_1} = P_2 - P_1 = (x_2 - x_1, y_2 - y_1, z_2 - z_1)$
     - $\mathbf{v_2} = P_3 - P_1 = (x_3 - x_1, y_3 - y_1, z_3 - z_1)$
2. **Compute the Cross Product**: The normal vector $\mathbf{n}$ can be obtained by taking the cross product of $\mathbf{v_1}$ and $\mathbf{v_2}$:

$$\mathbf{n} = \mathbf{v_1} \times \mathbf{v_2} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_2 - x_1 & y_2 - y_1 & z_2 - z_1 \\ x_3 - x_1 & y_3 - y_1 & z_3 - z_1 \end{vmatrix}$$

Once we have the normal vector $\mathbf{n} = (a, b, c)$, we can use one of the points (for instance, P_1) to write the equation of the plane in point-normal form:

$$a(x - x_1) + b(y - y_1) + c(z - z_1) = 0$$

Example 5.7. Find the equation of the plane passing through the points $P_1(1, 2, 3)$, $P_2(4, 5, 6)$, and $P_3(1, 4, 5)$.

[78]:
```
# Import the SymPy library for symbolic mathematics
import sympy as sp

# Define the points A, B, and C in 3D space
A = sp.Matrix([1, 2, 3])  # Point A with coordinates (1, 2, 3)
B = sp.Matrix([4, 5, 6])  # Point B with coordinates (4, 5, 6)
C = sp.Matrix([1, 4, 5])  # Point C with coordinates (1, 4, 5)

# Create vectors AB and AC
AB = B - A  # Vector AB represents the direction from point A to point B
AC = C - A  # Vector AC represents the direction from point A to point C

# Calculate the normal vector using the cross product
normal_vector = AB.cross(AC)  # The cross product of AB and AC gives the normal␣
 ↪vector to the plane defined by points A, B, and C
print('normal_vector =', normal_vector)  # Print the normal vector for␣
 ↪verification

# Create symbols for x, y, z
x, y, z = sp.symbols('x y z')  # These are symbolic variables that will␣
 ↪represent the coordinates of points in the plane

# Plane equation: n_x * (x - x_0) + n_y * (y - y_0) + n_z * (z - z_0) = 0
# This is the formula for a plane where (n_x, n_y, n_z) is the normal vector␣
 ↪and (x_0, y_0, z_0) is a point on the plane
# Create the plane equation using point A
plane_eq_A = normal_vector[0] * (x - A[0]) + normal_vector[1] * (y - A[1]) +␣
 ↪normal_vector[2] * (z - A[2])

# Create the plane equation using point B
plane_eq_B = normal_vector[0] * (x - B[0]) + normal_vector[1] * (y - B[1]) +␣
 ↪normal_vector[2] * (z - B[2])

# Create the plane equation using point C
plane_eq_C = normal_vector[0] * (x - C[0]) + normal_vector[1] * (y - C[1]) +␣
 ↪normal_vector[2] * (z - C[2])

# Display the equation of the plane
plane_eq_simplified_A = sp.simplify(plane_eq_A)  # Simplify the plane equation␣
 ↪using point A for clearer output
plane_eq_simplified_B = sp.simplify(plane_eq_B)  # Simplify the plane equation␣
 ↪using point B
plane_eq_simplified_C = sp.simplify(plane_eq_C)  # Simplify the plane equation␣
 ↪using point C

# Print the equations of the plane derived from each point
```

```
print(f"Equation of the plane using point A: {plane_eq_simplified_A} = 0")  #␣
 ↪Display the equation of the plane using point A
print(f"Equation of the plane using point B: {plane_eq_simplified_B} = 0")  #␣
 ↪Display the equation of the plane using point B
print(f"Equation of the plane using point C: {plane_eq_simplified_C} = 0")  #␣
 ↪Display the equation of the plane using point C
```

```
normal_vector = Matrix([[0], [-6], [6]])
Equation of the plane using point A: -6*y + 6*z - 6 = 0
Equation of the plane using point B: -6*y + 6*z - 6 = 0
Equation of the plane using point C: -6*y + 6*z - 6 = 0
```

5.5.4 Distance Between Two Lines

In three-dimensional space, the distance between two lines can be determined when the lines are skew (i.e., they do not intersect and are not parallel). The method involves using vector algebra to find the shortest distance between the two lines.

Consider two lines defined by their parametric equations: $\mathbf{r_1} = \mathbf{a_1} + t\mathbf{b_1}, \mathbf{r_2} = \mathbf{a_2} + s\mathbf{b_2}$ where $\mathbf{a_1}$ and $\mathbf{a_2}$ are points on lines 1 and 2. While $\mathbf{b_1}$ and $\mathbf{b_2}$ are direction vectors of lines 1 and 2, and t and s are parameters.

The distance D between the two lines can be computed using the following formula:

$$D = \frac{|(\mathbf{a_2} - \mathbf{a_1}) \cdot (\mathbf{b_1} \times \mathbf{b_2})|}{|\mathbf{b_1} \times \mathbf{b_2}|}$$

Example 5.8. Find the distance between the two lines with the following equations:

$$\mathbf{r_1} = (1, 2, 3) + t(4, 6, 5)$$
$$\mathbf{r_2} = (7, 8, 9) + s(1, 0, -1)$$

[79]:
```
# Import the SymPy library for symbolic mathematics
import sympy as sp

# Define points on the lines and direction vectors
p1 = sp.Matrix([1, 2, 3])  # Point on Line 1 with coordinates (1, 2, 3)
d1 = sp.Matrix([4, 6, 5])   # Direction vector for Line 1, indicating its␣
 ↪direction in space

p2 = sp.Matrix([7, 8, 9])  # Point on Line 2 with coordinates (4, 0, 2)
d2 = sp.Matrix([1, 0, -1])   # Direction vector for Line 2, indicating its␣
 ↪direction in space

# Calculate the vector between the two points
vector_between = p2 - p1  # This vector points from p1 to p2

# Calculate the cross product of the direction vectors
```

```
cross_product = d1.cross(d2)  # The cross product gives a vector that is␣
↪perpendicular to both direction vectors

# Calculate the distance between the two lines
# The formula used here is: Distance = |(P2 - P1) · (d1 x d2)| / ||(d1 x d2)||
# where (P2 - P1) is the vector between the two points and (d1 x d2) is the␣
↪cross product of the direction vectors
distance = sp.Abs(vector_between.dot(cross_product)) / cross_product.norm()  #␣
↪Calculate the distance

# Display the results
print(f"Distance between the lines: {distance} = {distance.evalf()}")  # Print␣
↪a message indicating what is being displayed
display(distance)  # Display the calculated distance using Jupyter's display␣
↪function
```

```
Distance between the lines: 6*sqrt(17)/17 = 1.45521375021800
```

$$\frac{6\sqrt{17}}{17}$$

5.5.5 Distance from a Point to a Plane

In three-dimensional space, the distance from a point to a plane can be calculated using the equation of the plane and the coordinates of the point.

Let the equation of the plane be given in the standard form: $ax + by + cz + d = 0$. The distance D from the point $P(x_0, y_0, z_0)$ to the plane can be calculated using the following formula:

$$D = \frac{|ax_0 + by_0 + cz_0 + d|}{\sqrt{a^2 + b^2 + c^2}}$$

Example 5.9. Find the distance from the point $P(1, 2, 3)$ to the plane $2x - 3y + 4z - 10 = 0$.

[80]:
```
# Import the SymPy library for symbolic mathematics
import sympy as sp

# Distance from a point P0 to a plane
# Define the point P0 (x0, y0, z0) and the plane represented by the equation Ax␣
↪+ By + Cz + D = 0
point = sp.Matrix([1, 2, 3])  # Point P0 with coordinates (1, 2, 3). This point␣
↪should not lie on the plane, otherwise the distance will be zero
A, B, C, D = 2, -3, 4, -10  # Coefficients of the plane equation

# Extract the coordinates of point P0
x0, y0, z0 = point[0], point[1], point[2]  # Assign the coordinates of point P0␣
↪to variables

# Calculate the distance from the point to the plane using the first method
```

```
# The formula for the distance from a point to a plane is:
# Distance = |Ax0 + By0 + Cz0 + D| / sqrt(A^2 + B^2 + C^2)
distance = sp.Abs(A*x0 + B*y0 + C*z0 + D) / sp.sqrt(A**2 + B**2 + C**2)

# Display the results of the first method
print("Display the results of the first method")
print(f"Distance from point {point} to the plane {A}x + {B}y + {C}z + {D} = 0:")
print(f"Distance: {distance} = {distance.evalf()}")  # Print the calculated␣
 ↪distance
display(distance)  # Display the distance using Jupyter's display function

# Calculate the distance from the point to the plane using the second method
# Choose a point P1 that lies on the plane. Any point satisfying Ax1 + By1 +␣
 ↪Cz1 + D = 0 will work
# Here, we take an arbitrary point (5, 0, 0) that lies on the plane
x1, y1, z1 = 5, 0, 0  # Coordinates of point P1
vector_P1P0 = sp.Matrix([x0 - x1, y0 - y1, z0 - z1])  # Vector from P1 to P0
normal_vector = sp.Matrix([A, B, C])  # Normal vector of the plane represented␣
 ↪by coefficients (A, B, C)

# Calculate the distance using the formula:
# Distance = |(P1P0) · n| / ||n||
# where (P1P0) is the vector from point P1 to point P0 and n is the normal␣
 ↪vector
distance = sp.Abs(vector_P1P0.dot(normal_vector)) / normal_vector.norm()  # Dot␣
 ↪product of the vector and normal vector divided by the norm of the normal␣
 ↪vector

# Display the results of the second method
print("\nDisplay the results of the second method")
print(f"Distance from point {point} to the plane {A}x + {B}y + {C}z + {D} = 0:")
print(f"Distance: {distance} = {distance.evalf()} ")  # Print the calculated␣
 ↪distance using the second method
display(distance)  # Display the distance using Jupyter's display function
```

```
Display the results of the first method
Distance from point Matrix([[1], [2], [3]]) to the plane 2x + -3y + 4z + -10 =
0:
Distance: 2*sqrt(29)/29 = 0.371390676354104
```

$$\frac{2\sqrt{29}}{29}$$

```
Display the results of the second method
Distance from point Matrix([[1], [2], [3]]) to the plane 2x + -3y + 4z + -10 =
0:
Distance: 2*sqrt(29)/29 = 0.371390676354104
```

$$\frac{2\sqrt{29}}{29}$$

5.5.6 Distance Between Two Planes

In three-dimensional space, the distance between two parallel planes can be determined using vector algebra and the equation of the planes. The method involves finding the perpendicular distance between the two planes, which is the shortest distance between them.

Consider two parallel planes defined by the equations:

$$Ax + By + Cz = D_1$$

$$Ax + By + Cz = D_2$$

The distance d between two parallel planes can be computed using the following formula:

$$d = \frac{|D_2 - D_1|}{\sqrt{A^2 + B^2 + C^2}}$$

The distance between two parallel planes can be calculated using the formula above, where the difference in their constant terms and the magnitude of the normal vector are key to determining the shortest distance between them.

Alternatively, pick a point in one of the given planes, then calculate distance of the point to the other plane

Example 5.10. Find the distance between the two parallel planes with the following equations:

$$x + 3y - 2z = 4 \tag{5.1}$$
$$x + 3y - 2z = 10 \tag{5.2}$$

[81]:
```
# Importing the necessary function from the sympy library
import sympy as sp

# Define the variables for the coefficients of the planes
A, B, C, D1, D2 = sp.symbols('A B C D1 D2')

# The normal vector to the planes is (A, B, C), so we can directly use these␣
 ↪values
# In the example: A = 1, B = 3, C = -2
A_value = 1
B_value = 3
C_value = -2

# Define the constant terms for the planes: D1 = 4 (for plane 1), D2 = 10 (for␣
 ↪plane 2)
D1_value = 4
```

```
D2_value = 10

# Now, we apply the formula to calculate the distance between the two planes
# The formula is: d = |D2 - D1| / sqrt(A^2 + B^2 + C^2)

# First, compute the absolute difference in D values
difference_in_D = sp.Abs(D2_value - D1_value)

# Compute the magnitude of the normal vector (sqrt(A^2 + B^2 + C^2))
normal_magnitude = sp.sqrt(A_value**2 + B_value**2 + C_value**2)

# Now calculate the distance between the planes
distance = difference_in_D / normal_magnitude

# Print the result
print(f"The distance between the two parallel planes is: {distance:.2f} units")
```

```
The distance between the two parallel planes is: 1.60 units
```

5.6 Surfaces in Space

In mathematics, a **surface** is a two-dimensional manifold that exists in three-dimensional space. Surfaces can be defined in various ways, and they play a crucial role in geometry, physics, and engineering. Here we are discussing quadratic surface in form:

$$ax^2 + by^2 + cz^2 + dxy + eyz + fxz + gx + hy + jz + k = 0$$

where at least one of a, b, c, e, f, g is non-zero. We will focus on three special shapes: Ellipsoids, Paraboloids, and Elliptic Cones.

Drawing Traces

1. Fix a Variable: Choose a specific value for one of the variables (x, y, or z). This will reduce the three-dimensional surface equation to a two-dimensional plane curve.
2. Solve for the Remaining Variables: Substitute the fixed variable value into the surface equation and solve for the remaining variables.
3. Plot the Plane Curve: Plot the resulting two-dimensional curve in the appropriate plane (e.g., xy-plane, yz-plane, or xz-plane).

Drawing Surfaces

1. Choose a Coordinate System: Decide whether to use Cartesian coordinates, parametric equations, or another coordinate system to represent the surface.
2. Define the Surface Equation: Write the surface equation in the chosen coordinate system.
3. Plot the Surface: Use a plotting tool or software to visualize the surface. This can be done directly by solving the surface equation or by using parametric equations to represent the surface.

The following subsections provide examples of the procedure.

5.6.1 Ellipsoids

An **ellipsoid** is a three-dimensional geometric shape that resembles a stretched or compressed sphere. Mathematically, it is defined as the set of all points (x, y, z) in space that satisfy the equation:

$$\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} + \frac{(z-l)^2}{c^2} = 1$$

where (h, k, l) is the center of the ellipsoid, and a, b, and c are the semi-principal axes along the x, y, and z axes, respectively.

Sketching an Ellipsoid

To visualize an ellipsoid, it is useful to sketch its cross-sections on the coordinate planes: xy, yz, and xz.

Sketching in the xy-Plane

To sketch the ellipsoid in the xy-plane, set $z = l$ (the center's z-coordinate). The equation simplifies to:

$$\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} = 1$$

This equation represents an ellipse centered at (h, k) with semi-major axis a and semi-minor axis b. . To draw the cross section, we can either define a parameterization of the curve:

$$x = h + a\cos(t) \quad y = k + b\sin(t), \quad 0 \le t < 2\pi.$$

Alternatively we can directly plot using `implicit_plot`. Analogous procedure hold for yz, xz plane.

To draw the surface of ellipsoid, we can either plot the two function:

$$z = l \pm c\sqrt{1 - \frac{(x-h)^2}{a^2} - \frac{(y-k)^2}{b^2}}$$

or plot the parametric equation:

$$\begin{aligned} x &= h + a\sin(s)\cos(t) \\ y &= k + b\sin(s)\sin(t) \\ z &= l + c\cos(s) \end{aligned}$$

Example 5.11. Given elipsoid equation

$$\frac{(x-3)^2}{100} + \frac{(y-2)^2}{225} + \frac{(z-1)^2}{9} = 1$$

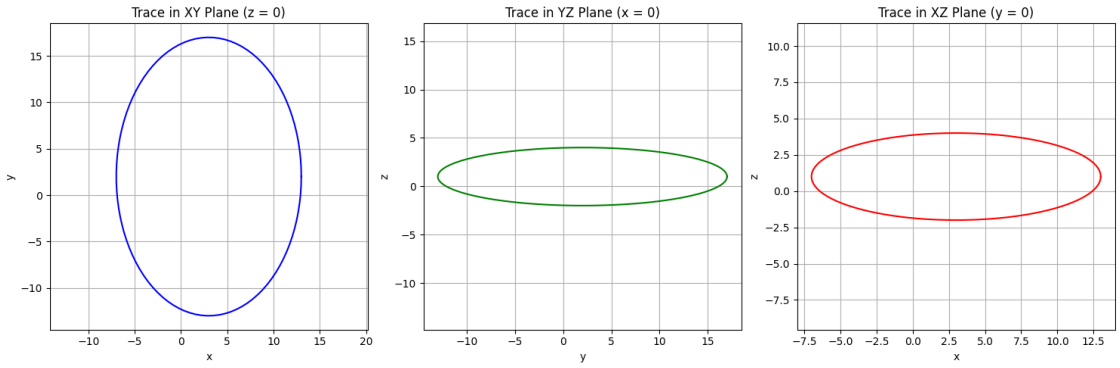
draw the surface and sketch the ellipsoid in (xy, yz, xz) planes .

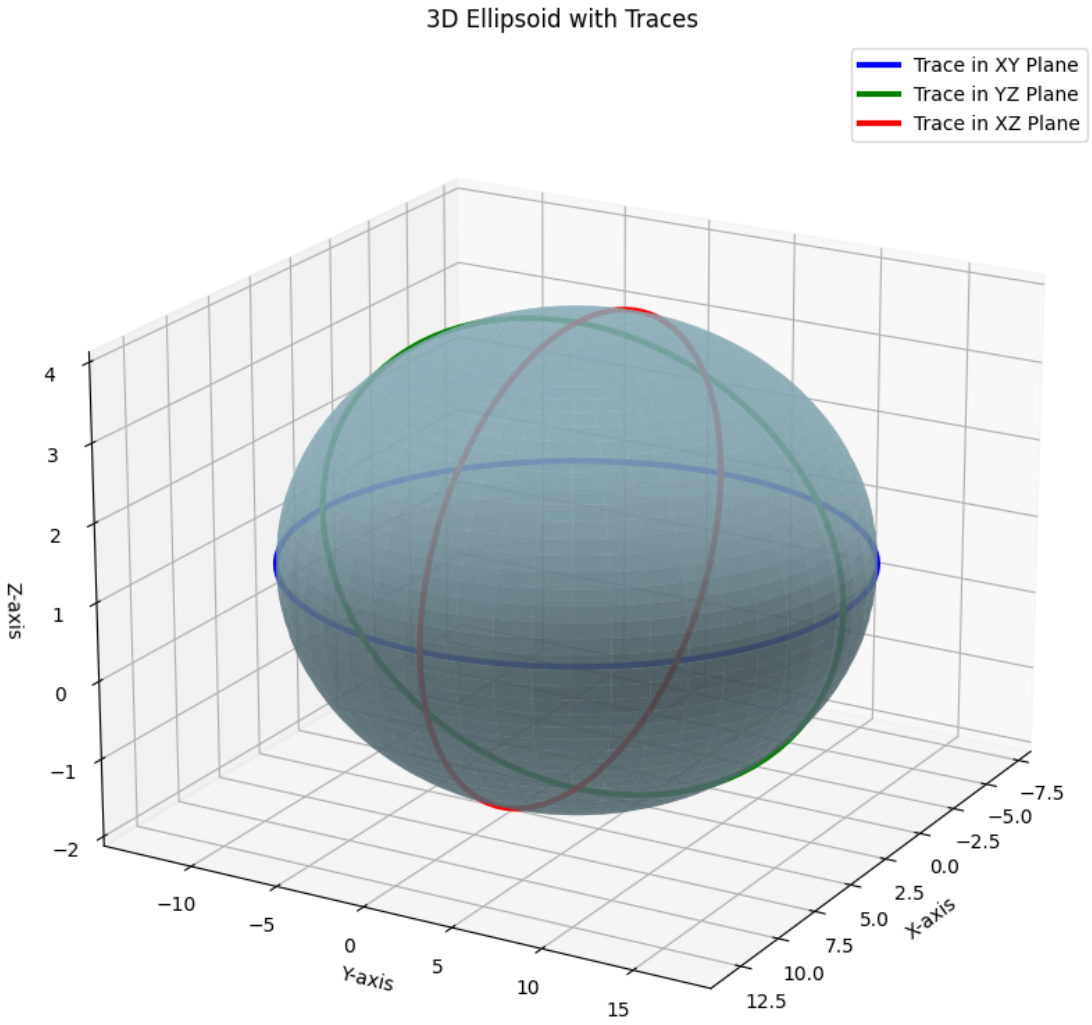
[82]:
```
# Import necessary libraries for mathematical operations and plotting
import numpy as np  # For numerical calculations
import matplotlib.pyplot as plt  # For creating 2D plots
```

```
from mpl_toolkits.mplot3d import Axes3D  # For creating 3D plots
import sympy as smp

# Parameters for the ellipsoid
### >>> Modify input parameter here
center = np.array([3, 2, 1])  # Define the center of the ellipsoid (h, k, l)
axes = np.array([10, 15, 3])     # Define the semi-axis lengths (a, b, c)
### <<< Modify input parameter here


# Generate points for the traces of the ellipsoid
theta = np.linspace(0, 2 * np.pi, 100)  # Create 100 points for angle θ ranging␣
 ↪from 0 to 2π

# Trace in the XY plane (where z = 0)
x_xy = center[0] + axes[0] * np.cos(theta)  # Calculate X coordinates for the␣
 ↪trace
y_xy = center[1] + axes[1] * np.sin(theta)  # Calculate Y coordinates for the␣
 ↪trace

# Trace in the YZ plane (where x = 0)
y_yz = center[1] + axes[1] * np.cos(theta)  # Calculate Y coordinates for the␣
 ↪trace
z_yz = center[2] + axes[2] * np.sin(theta)  # Calculate Z coordinates for the␣
 ↪trace

# Trace in the XZ plane (where y = 0)
x_xz = center[0] + axes[0] * np.cos(theta)  # Calculate X coordinates for the␣
 ↪trace
z_xz = center[2] + axes[2] * np.sin(theta)  # Calculate Z coordinates for the␣
 ↪trace
x,y=smp.symbols("x y", real=True)
# Create 2D plots for the traces in different planes
fig, axs = plt.subplots(1, 3, figsize=(15, 5))  # Create a figure with a 1x3␣
 ↪grid of subplots

# Plot the trace in the XY Plane
axs[0].plot(x_xy, y_xy, color='b')  # Plot the trace with a blue line
axs[0].set_title('Trace in XY Plane (z = 0)')  # Set the title for this subplot
axs[0].set_xlabel('x')  # Label for the x-axis
axs[0].set_ylabel('y')  # Label for the y-axis
axs[0].axis('equal')  # Ensure equal scaling for both axes
axs[0].grid(True)  # Enable the grid for better visualization

# Plot the trace in the YZ Plane
axs[1].plot(y_yz, z_yz, color='g')  # Plot the trace with a green line
```

```
axs[1].set_title('Trace in YZ Plane (x = 0)')  # Set the title for this subplot
axs[1].set_xlabel('y')  # Label for the y-axis
axs[1].set_ylabel('z')  # Label for the z-axis
axs[1].axis('equal')  # Ensure equal scaling for both axes
axs[1].grid(True)  # Enable the grid for better visualization

# Plot the trace in the XZ Plane
axs[2].plot(x_xz, z_xz, color='r')  # Plot the trace with a red line
axs[2].set_title('Trace in XZ Plane (y = 0)')  # Set the title for this subplot
axs[2].set_xlabel('x')  # Label for the x-axis
axs[2].set_ylabel('z')  # Label for the z-axis
axs[2].axis('equal')  # Ensure equal scaling for both axes
axs[2].grid(True)  # Enable the grid for better visualization


# Show the 2D plots
plt.tight_layout()  # Adjust the layout of the subplots for better spacing
plt.show()  # Display the 2D plots

# Create a grid of points for the ellipsoid in 3D
s = np.linspace(0, 2 * np.pi, 100)  # Create 100 points for the azimuthal angle
t = np.linspace(0, np.pi, 100)  # Create 100 points for the polar angle

# Parametric equations for the ellipsoid surface
x = center[0] + axes[0] * np.outer(np.cos(s), np.sin(t))  # Calculate X␣
 ↪coordinates
y = center[1] + axes[1] * np.outer(np.sin(s), np.sin(t))  # Calculate Y␣
 ↪coordinates
z = center[2] + axes[2] * np.outer(np.ones(np.size(s)), np.cos(t))  # Calculate␣
 ↪Z coordinates

# Create a 3D plot for the ellipsoid
fig = plt.figure(figsize=(10, 10))  # Create a new figure for the 3D plot
ax = fig.add_subplot(111, projection='3d')  # Add a 3D subplot

# Plot the ellipsoid surface
ax.plot_surface(x, y, z, color='lightblue', alpha=0.7)  # Plot the surface of␣
 ↪the ellipsoid with transparency

# Plot the traces in the 3D ellipsoid
ax.plot(x_xy, y_xy, zs=center[2], color='b', linewidth=3, label='Trace in XY␣
 ↪Plane')  # Add XY trace to the 3D plot
ax.plot(np.zeros_like(y_yz) + center[0], y_yz, z_yz, color='g', linewidth=3,␣
 ↪label='Trace in YZ Plane')  # Add YZ trace to the 3D plot
ax.plot(x_xz, np.zeros_like(z_xz) + center[1], z_xz, color='r', linewidth=3,␣
 ↪label='Trace in XZ Plane')  # Add XZ trace to the 3D plot
```

```
# Set labels and title for the 3D plot
ax.set_xlabel('X-axis')  # Label for the x-axis
ax.set_ylabel('Y-axis')  # Label for the y-axis
ax.set_zlabel('Z-axis')  # Label for the z-axis
ax.set_title('3D Ellipsoid with Traces')  # Title for the 3D plot
ax.legend()  # Show the legend for the traces
ax.view_init(elev=20, azim=30)  # Set viewing angle for better visualization of␣
 ↪the ellipsoid

# Show the 3D plot
plt.show()  # Display the 3D plot
```





5.6.2 Paraboloids

A **paraboloid** is a three-dimensional surface that is defined by a quadratic equation in three variables. There are two types of paraboloids: **elliptic paraboloids** and **hyperbolic paraboloids**.

Types of Paraboloids

1. Elliptic Paraboloid

An **elliptic paraboloid** is shaped like an upward or downward opening dish and is defined by the equation:

$$z = \frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} + l$$

where (h, k, l) is the vertex of the paraboloid and a and b control the curvature along the x and y axes, respectively.

The cross-sections of an elliptic paraboloid parallel to the xy-plane are ellipses, while cross-sections parallel to planes that include the z-axis are parabolas.

2. Hyperbolic Paraboloid

A **hyperbolic paraboloid**, also known as a saddle surface, is defined by the equation:

$$z = \frac{(x-h)^2}{a^2} - \frac{(y-k)^2}{b^2} + l$$

where (h, k, l) is the vertex of the paraboloid and a and b control the curvature along the x and y axes, respectively.

The cross-sections of a hyperbolic paraboloid parallel to the xy-plane are hyperbolas, while those parallel to the yz- or xz-planes are parabolas.

Sketching a Paraboloid

To visualize a paraboloid, we can sketch its cross-sections on the coordinate planes: xy, yz, and xz.

1. Sketching in the xy-Plane (Elliptic Paraboloid)

For an elliptic paraboloid defined by:

$$z = \frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} + l$$

Setting $z = c$ (a constant) allows us to visualize the cross-section as an ellipse:

$$\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} = c - l$$

2. Sketching in the yz-Plane

For an elliptic paraboloid, setting $x = h$ gives:

$$z = \frac{(y-k)^2}{b^2} + l$$

This represents a parabola opening in the positive z-direction. Analogous equation holds for sketching in xz-plane. We can parameterize the equation to draw the sketch.

To draw the surface, we should create x and y mesh and then plot the function:

$$z = \frac{(x-h)^2}{a^2} \pm \frac{(y-k)^2}{b^2} + l$$

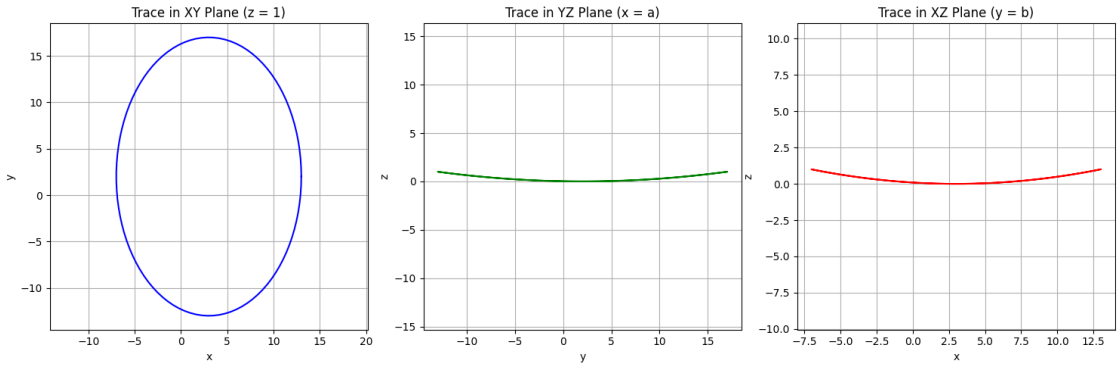
Example 5.12. Sketch the paraboloid defined by the equation

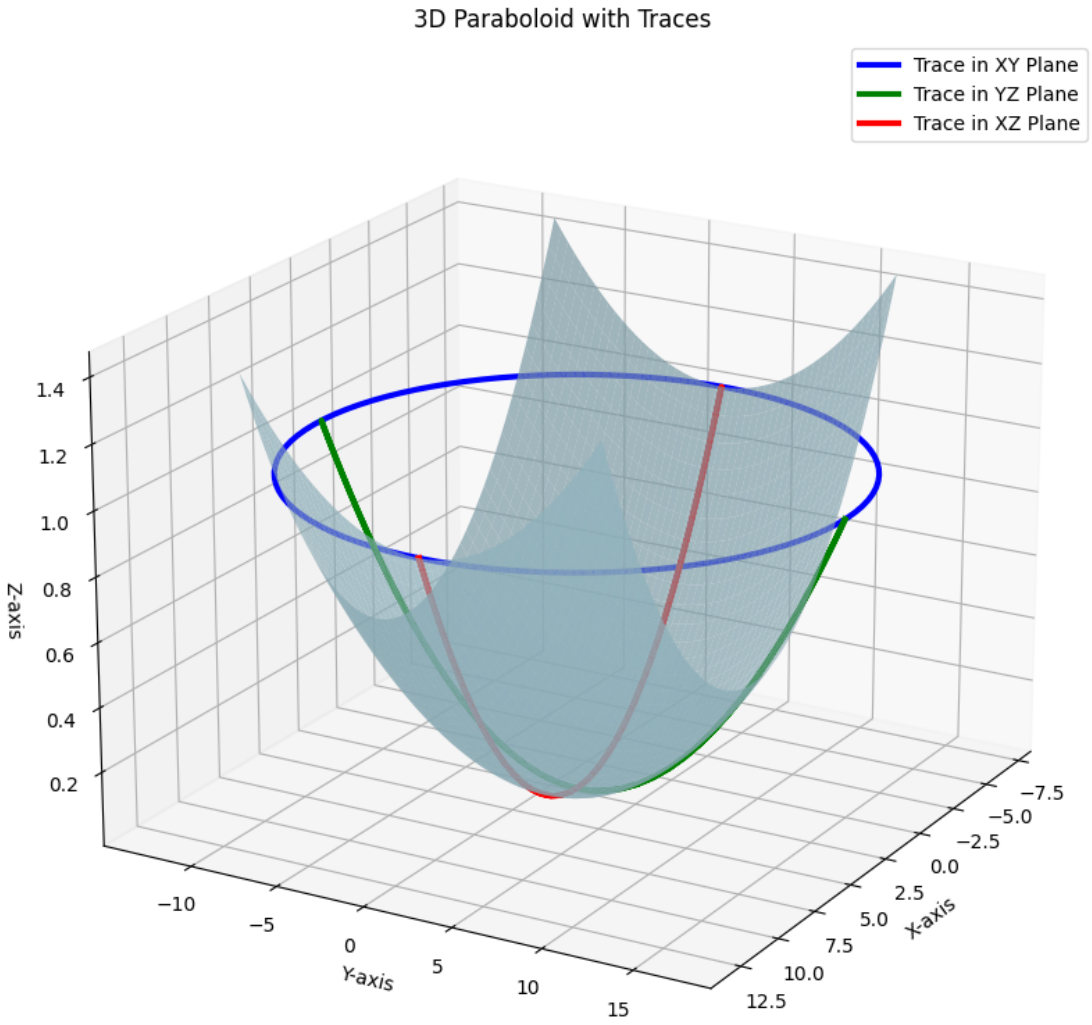
$$\frac{(x-3)^2}{100} + \frac{(y-2)^2}{225} = z$$

[83]:
```
# Import necessary libraries for numerical calculations and plotting
import numpy as np  # For numerical operations
import matplotlib.pyplot as plt  # For creating 2D plots
from mpl_toolkits.mplot3d import Axes3D  # For creating 3D plots

# Parameters for the paraboloid
### >>> Modify input parameter here
vertex = np.array([3, 2, 0])  # Define the vertex of the paraboloid␣
↪(coordinates h, k, l)
widths = np.array([10, 15])      # Define the widths of the paraboloid along x␣
↪(a) and y (b)
### <<< Modify input parameter here

# Create a grid of points for the paraboloid
x = np.linspace(vertex[0] - 10, vertex[0] + 10, 100)  # Generate X values␣
↪around the vertex
y = np.linspace(vertex[1] - 10, vertex[1] + 10, 100)  # Generate Y values␣
↪around the vertex
x, y = np.meshgrid(x, y)  # Create a meshgrid (grid of x,y coordinates)

# Calculate z values based on the paraboloid equation
# z = c + ((x - a)^2 / d^2) + ((y - b)^2 / e^2)
z = vertex[2] + ((x - vertex[0])**2 / widths[0]**2) + ((y - vertex[1])**2 /␣
↪widths[1]**2)  # Calculate Z values

# Create traces in the 2D planes
theta = np.linspace(0, 2 * np.pi, 100)  # Create 100 points for angle θ ranging␣
↪from 0 to 2π

# Trace in the XY plane (where z = constant)
z_xy = 1  # Set Z value for the trace in the XY plane (this is a constant)
x_xy = vertex[0] + widths[0] * np.cos(theta)  # Calculate X coordinates for the␣
↪trace
y_xy = vertex[1] + widths[1] * np.sin(theta)  # Calculate Y coordinates for the␣
↪trace

# Trace in the YZ plane (where x = constant)
x_yz = vertex[0]  # Set X value for the trace in the YZ plane (constant)
y_yz = vertex[1] + widths[1] * np.cos(theta)  # Calculate Y coordinates for the␣
↪trace
z_yz = vertex[2] + ((y_yz - vertex[1])**2 / widths[1]**2)  # Calculate Z␣
↪coordinates for the trace

# Trace in the XZ plane (where y = constant)
y_xz = vertex[1]  # Set Y value for the trace in the XZ plane (constant)
```

```
x_xz = vertex[0] + widths[0] * np.cos(theta)  # Calculate X coordinates for the␣
↪trace
z_xz = vertex[2] + ((x_xz - vertex[0])**2 / widths[0]**2)  # Calculate Z␣
↪coordinates for the trace

# Create the 2D plots for the traces in different planes
fig, axs = plt.subplots(1, 3, figsize=(15, 5))  # Create a figure with a 1x3␣
↪grid of subplots

# Plot the trace in the XY Plane
axs[0].plot(x_xy, y_xy, color='b')  # Plot the trace with a blue line
axs[0].set_title('Trace in XY Plane (z = 1)')  # Set the title for this subplot
axs[0].set_xlabel('x')  # Label for the x-axis
axs[0].set_ylabel('y')  # Label for the y-axis
axs[0].axis('equal')  # Ensure equal scaling for both axes
axs[0].grid(True)  # Enable grid for better visualization

# Plot the trace in the YZ Plane
axs[1].plot(y_yz, z_yz, color='g')  # Plot the trace with a green line
axs[1].set_title('Trace in YZ Plane (x = a)')  # Set the title for this subplot
axs[1].set_xlabel('y')  # Label for the y-axis
axs[1].set_ylabel('z')  # Label for the z-axis
axs[1].axis('equal')  # Ensure equal scaling for both axes
axs[1].grid(True)  # Enable grid for better visualization

# Plot the trace in the XZ Plane
axs[2].plot(x_xz, z_xz, color='r')  # Plot the trace with a red line
axs[2].set_title('Trace in XZ Plane (y = b)')  # Set the title for this subplot
axs[2].set_xlabel('x')  # Label for the x-axis
axs[2].set_ylabel('z')  # Label for the z-axis
axs[2].axis('equal')  # Ensure equal scaling for both axes
axs[2].grid(True)  # Enable grid for better visualization

# Show the 2D plots
plt.tight_layout()  # Adjust the layout of the subplots for better spacing
plt.show()  # Display the 2D plots

# Create a 3D plot for the paraboloid
fig = plt.figure(figsize=(10, 10))  # Create a new figure for the 3D plot
ax = fig.add_subplot(111, projection='3d')  # Add a 3D subplot

# Plot the paraboloid surface
ax.plot_surface(x, y, z, color='lightblue', alpha=0.7)  # Plot the surface with␣
↪some transparency

# Plot traces in the 3D paraboloid
```

```
ax.plot(x_xy, y_xy, zs=z_xy, color='b', linewidth=3, label='Trace in XY Plane')␣
 ↪ # Add XY trace to the 3D plot
ax.plot(np.full_like(y_yz, x_yz), y_yz, z_yz, color='g', linewidth=3,␣
 ↪label='Trace in YZ Plane')  # Add YZ trace to the 3D plot
ax.plot(x_xz, np.full_like(z_xz, y_xz), z_xz, color='r', linewidth=3,␣
 ↪label='Trace in XZ Plane')  # Add XZ trace to the 3D plot

# Set labels and title for the 3D plot
ax.set_xlabel('X-axis')  # Label for the x-axis
ax.set_ylabel('Y-axis')  # Label for the y-axis
ax.set_zlabel('Z-axis')  # Label for the z-axis
ax.set_title('3D Paraboloid with Traces')  # Title for the 3D plot
ax.legend()  # Show the legend for the traces
ax.view_init(elev=20, azim=30)  # Set viewing angle for better visualization

# Show the 3D plot
plt.show()  # Display the plot
```





5.6.3 Elliptic Cones

An **elliptic cone** is a three-dimensional geometric figure that is formed by extending a circle or an ellipse along a vertical axis. It can be visualized as a cone with an elliptical base. The equation of an elliptic cone is given in its standard form, which describes its shape and orientation in space.

Definition of an Elliptic Cone

The equation of an elliptic cone can be expressed as:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = \frac{z^2}{c^2}$$

where: - a and b are the semi-major and semi-minor axes of the base ellipse, respectively. - c is a scaling

factor that determines the "height" of the cone.

The apex of the cone is located at the origin $(0, 0, 0)$, and as z increases, the cross-section of the cone resembles an ellipse that becomes larger.

Cross-Sections of an Elliptic Cone

Cross-sections of an elliptic cone can be analyzed by setting different values of z:

1. Cross-section in the xy-Plane

When $z = k$, the equation becomes:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = \frac{z^2}{c^2}$$

This represents an ellipse.

2. Cross-section in the yz-Plane

To find the cross-section of the elliptic cone in the yz-plane, we set $x = 0$ in the equation of the cone:

$$\frac{y^2}{b^2} = \frac{z^2}{c^2}$$

This represents a pair of straight lines, defined by:

$$y = \pm\frac{b}{c}z$$

The resulting sketch of the elliptic cone's cross-section in the yz-plane consists of two lines that extend from the apex at the origin, diverging as they move away from the apex.

Example 5.13. Given eliptic cone equation

$$\frac{x^2}{16} + \frac{y^2}{49} = \frac{z^2}{100}$$

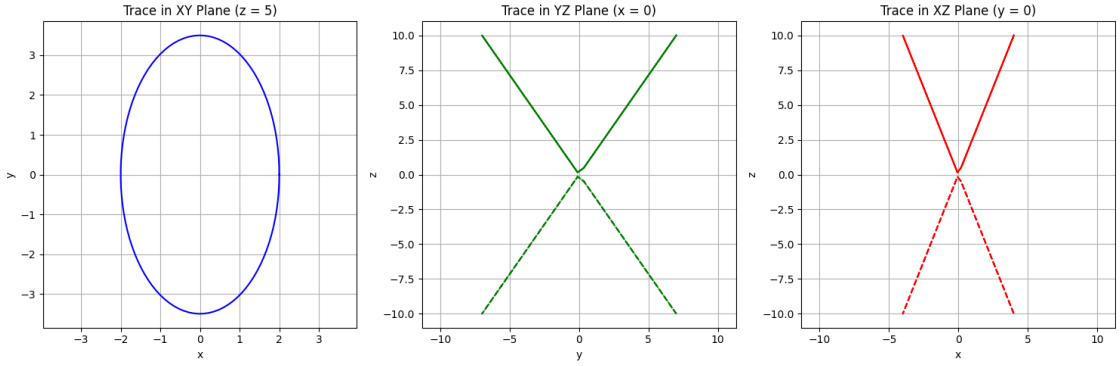
Plot the traces of the elliptic cone in the XY, YZ, and XZ planes. Then, create a 3D plot of the elliptic cone with the traces. create a 3D plot of the elliptic cone with the traces.

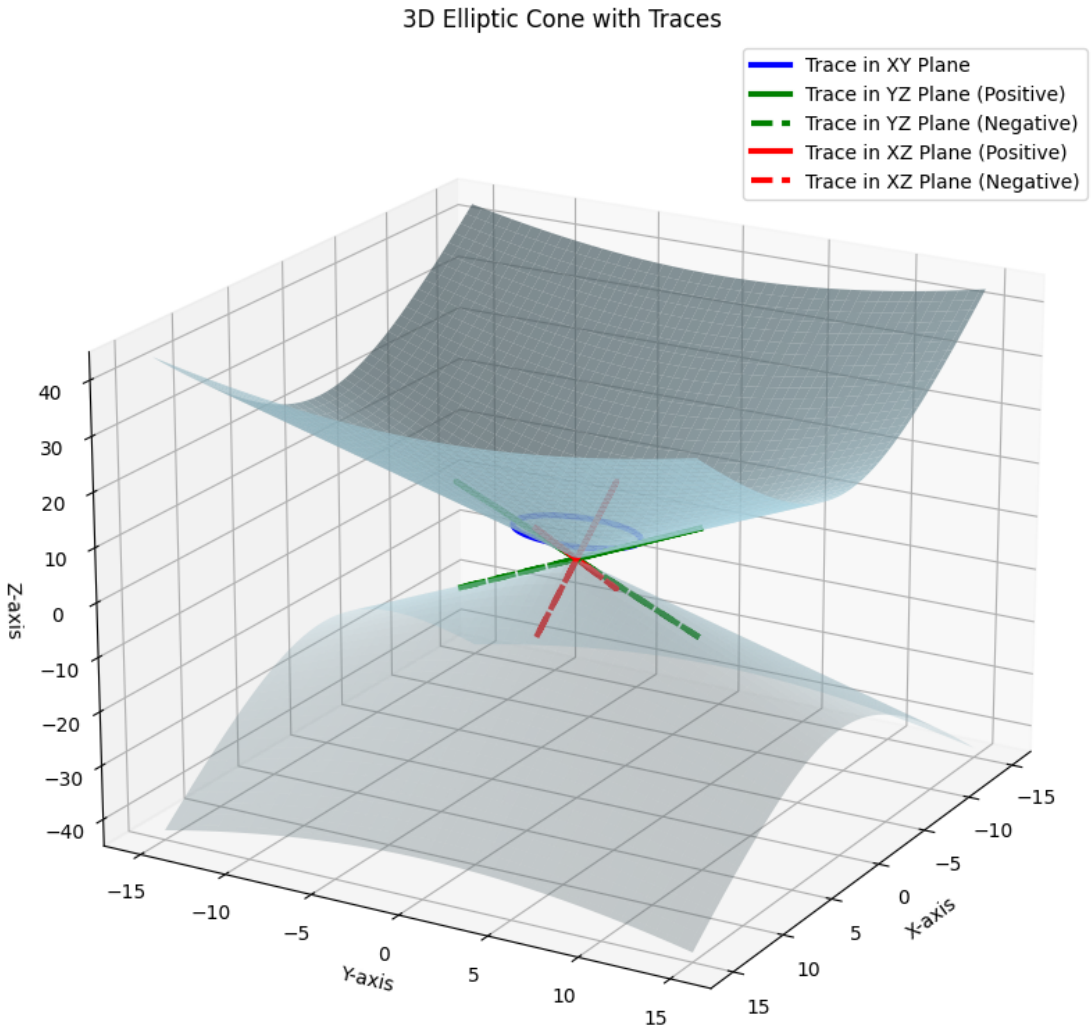
[84]:
```
# Import necessary libraries for numerical calculations and plotting
import numpy as np  # For numerical operations
import matplotlib.pyplot as plt  # For creating 2D plots
from mpl_toolkits.mplot3d import Axes3D  # For creating 3D plots

#### MODIFY THE PARAMETER HERE ####
# Parameters for the elliptic cone
vertex = np.array([0, 0, 0])  # Define the vertex of the cone at the origin (0,␣
↪0, 0)
a = 4  # Semi-axis length along x
b = 7  # Semi-axis length along y
c = 10  # Semi-axis length along z
```

```
#### MODIFY THE PARAMETER HERE ####




# Create traces in the 2D planes
theta = np.linspace(0, 2 * np.pi, 100)  # Create 100 points for angle θ ranging␣
↪from 0 to 2π

# Trace in the XY plane (where z = constant)
z_xy = 5  # Set Z value for the trace in the XY plane (constant)
x_xy = a * np.cos(theta) * (z_xy / c)  # Calculate X coordinates for the trace
y_xy = b * np.sin(theta) * (z_xy / c)  # Calculate Y coordinates for the trace

# Trace in the YZ plane (where x = constant)
x_yz = 0  # Set X value for the trace in the YZ plane (constant)
y_yz = b * np.cos(theta)  # Calculate Y coordinates for the trace
z_yz_pos = np.sqrt((x_yz**2 / a**2 + (y_yz**2) / b**2) * c**2)  # Positive Z␣
↪coordinates for the trace
z_yz_neg = -z_yz_pos  # Negative Z coordinates for the trace

# Trace in the XZ plane (where y = constant)
y_xz = 0  # Set Y value for the trace in the XZ plane (constant)
x_xz = a * np.cos(theta)  # Calculate X coordinates for the trace
z_xz_pos = np.sqrt(((x_xz**2 / a**2) + (y_xz**2 / b**2)) * c**2)  # Positive Z␣
↪coordinates for the trace
z_xz_neg = -z_xz_pos  # Negative Z coordinates for the trace

# Create the 2D plots for the traces in different planes
fig, axs = plt.subplots(1, 3, figsize=(15, 5))  # Create a figure with a 1x3␣
↪grid of subplots

# Plot the trace in the XY Plane
axs[0].plot(x_xy, y_xy, color='b')  # Plot the trace with a blue line
axs[0].set_title('Trace in XY Plane (z = 5)')  # Set the title for this subplot
axs[0].set_xlabel('x')  # Label for the x-axis
axs[0].set_ylabel('y')  # Label for the y-axis
axs[0].axis('equal')  # Ensure equal scaling for both axes
axs[0].grid(True)  # Enable grid for better visualization

# Plot the trace in the YZ Plane
axs[1].plot(y_yz, z_yz_pos, color='g')  # Plot the positive trace with a green␣
↪line
axs[1].plot(y_yz, z_yz_neg, color='g', linestyle='--')  # Plot the negative␣
↪trace with a dashed green line
axs[1].set_title('Trace in YZ Plane (x = 0)')  # Set the title for this subplot
```

```
axs[1].set_xlabel('y')  # Label for the y-axis
axs[1].set_ylabel('z')  # Label for the z-axis
axs[1].axis('equal')  # Ensure equal scaling for both axes
axs[1].grid(True)  # Enable grid for better visualization

# Plot the trace in the XZ Plane
axs[2].plot(x_xz, z_xz_pos, color='r')  # Plot the positive trace with a red␣
↪line
axs[2].plot(x_xz, z_xz_neg, color='r', linestyle='--')  # Plot the negative␣
↪trace with a dashed red line
axs[2].set_title('Trace in XZ Plane (y = 0)')  # Set the title for this subplot
axs[2].set_xlabel('x')  # Label for the x-axis
axs[2].set_ylabel('z')  # Label for the z-axis
axs[2].axis('equal')  # Ensure equal scaling for both axes
axs[2].grid(True)  # Enable grid for better visualization

# Show the 2D plots
plt.tight_layout()  # Adjust the layout of the subplots for better spacing
plt.show()  # Display the 2D plots

# Create a grid of points for the cone
x = np.linspace(-15, 15, 100)  # Generate X values
y = np.linspace(-15, 15, 100)  # Generate Y values
x, y = np.meshgrid(x, y)  # Create a meshgrid from x and y

# Calculate z values based on the elliptic cone equation
z = np.sqrt((x**2 / a**2 + y**2 / b**2) * c**2)  # Positive root for the upper␣
↪cone
z_neg = -z  # Negative root for the lower cone


# Create a 3D plot for the elliptic cone
fig = plt.figure(figsize=(10, 10))  # Create a new figure for the 3D plot
ax = fig.add_subplot(111, projection='3d')  # Add a 3D subplot

# Plot the cone surfaces
ax.plot_surface(x, y, z, color='lightblue', alpha=0.7)  # Plot the surface for␣
↪the upper cone
ax.plot_surface(x, y, z_neg, color='lightblue', alpha=0.3)  # Plot the surface␣
↪for the lower cone with transparency

# Plot traces in the 3D cone
ax.plot(x_xy, y_xy, zs=z_xy, color='b', linewidth=3, label='Trace in XY Plane')␣
↪ # Add XY trace to the 3D plot
ax.plot(np.full_like(y_yz, x_yz), y_yz, z_yz_pos, color='g', linewidth=3,␣
↪label='Trace in YZ Plane (Positive)')  # Add positive YZ trace to the 3D plot
```

```
ax.plot(np.full_like(y_yz, x_yz), y_yz, z_yz_neg, color='g', linestyle='--',␣
 ↪linewidth=3, label='Trace in YZ Plane (Negative)')  # Add negative YZ trace␣
 ↪to the 3D plot
ax.plot(x_xz, np.full_like(z_xz, y_xz), z_xz_pos, color='r', linewidth=3,␣
 ↪label='Trace in XZ Plane (Positive)')  # Add positive XZ trace to the 3D plot
ax.plot(x_xz, np.full_like(z_xz, y_xz), z_xz_neg, color='r', linestyle='--',␣
 ↪linewidth=3, label='Trace in XZ Plane (Negative)')  # Add negative XZ trace␣
 ↪to the 3D plot

# Set labels and title for the 3D plot
ax.set_xlabel('X-axis')  # Label for the x-axis
ax.set_ylabel('Y-axis')  # Label for the y-axis
ax.set_zlabel('Z-axis')  # Label for the z-axis
ax.set_title('3D Elliptic Cone with Traces')  # Title for the 3D plot
ax.legend()  # Show legend for the traces
ax.view_init(elev=20, azim=30)  # Set viewing angle for better visualization

# Show the 3D plot
plt.show()  # Display the plot
```




3D Elliptic Cone with Traces
Trace in XY Plane
Trace in YZ Plane (Positive)
Trace in YZ Plane (Negative)
Trace in XZ Plane (Positive)
Trace in XZ Plane (Negative)
Z-axis
40
30
20
10
0
−10
−20
−30
−40
Y-axis
−15
−10
−5
0
5
10
15
X-axis
−15
−10
−5
0
5
10
15